

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Веб-приложение для игры в шахматы с обучающими сценариями на платформе
VK Games**

Обучающийся: Шадрухин Александр Сергеевич

Квалификация: Бакалавриат

Факультет/институт/кластер: Факультет программной инженерии и компьютерной
техники

Направление подготовки: 09.03.04 Программная инженерия

Образовательная программа: Нейротехнологии и программирование

Группа: Р3425

Руководитель ВКР: Болдырева Елена Александровна

Обучающийся

Шадрухин Александр Сергеевич

Дата: 25.05.2026

ЗАДАНИЕ

Основные вопросы, подлежащие разработке:

Цель работы

Повышение эффективности обучения дебютам в шахматных партиях за счет использования веб-приложения с интерактивными сценариями обучения.

Техническое задание

1. Проведение анализа предметной области обучения шахматным дебютам, включая обзор существующих шахматных приложений и обучающих систем, выявление их преимуществ, ограничений и особенностей применения в контексте интерактивного обучения пользователей.
2. Проектирование архитектуры веб-приложения для обучения шахматным дебютам, включающей клиентскую часть, серверную часть на Java Spring Boot, базу данных PostgreSQL, модуль хранения пользовательского прогресса, REST API для обмена данными, а также взаимодействие с платформой VK Games.
3. Разработка игровой логики веб-приложения, включающей реализацию классических шахмат с корректной обработкой правил, ходов фигур, очередности ходов, игровых состояний и завершения партии.
4. Разработка модуля обучающих сценариев по шахматным дебютам, включающего набор дебютных сценариев, демонстрацию правильного разыгрывания позиции, самостоятельное повторение изученного дебюта, систему подсказок и пошаговое сопровождение пользователя при прохождении сценария.
5. Разработка модуля оценки уровня подготовки пользователя, включающего тест для определения начального уровня игрока, итоговый тест для проверки усвоения дебютного материала, а также расчет эффективности обучения на основе результатов тестирования и уровня подготовки пользователя.
6. Реализация механизма отслеживания прогресса пользователя, включающего хранение результатов прохождения обучающих сценариев, сохранение результатов тестов, формирование истории решенных задач и пройденных дебютов, а также возможность повторного прохождения ошибочных или незавершенных сценариев.
7. Разработка пользовательского интерфейса веб-приложения с использованием React, TypeScript и VKUI, обеспечивающего удобное взаимодействие с шахматной доской, обучающими сценариями, тестами, системой подсказок, панелью прогресса и историей прохождения.
8. Реализация элементов онлайн-взаимодействия пользователей, включая создание онлайн-партий, подключение к игровым комнатам, продолжение ранее начатых партий и передачу ходов между пользователями через WebSocket.

9. Проведение тестирования, отладки и развертывания веб-приложения, включая проверку корректности игровой логики, работы обучающих сценариев, сохранения прогресса, расчёта эффективности обучения, отображения интерфейса, функционирования REST API, WebSocket-взаимодействия и работы приложения на платформе VK Games. Исходный код клиентской и серверной частей планируется разместить в открытом репозитории.

Ожидаемый результат

Веб-приложение, реализованное на платформе VK Games, предоставляющее пользователям возможность играть в шахматы, изучать шахматные дебюты с помощью интерактивных сценариев, проходить тестирование начального уровня, выполнять итоговый тест и получать оценку эффективности обучения. Серверная часть приложения реализуется на Java Spring Boot, взаимодействие между клиентом и сервером организуется через REST API и WebSocket, а хранение пользовательского прогресса и результатов тестирования выполняется в базе данных PostgreSQL.

Выходной результат

1. Веб-приложение для игры в шахматы и обучения шахматным дебютам.
2. Интерактивные обучающие сценарии с пошаговым разбором.
3. Система подсказок и демонстрации правильного разыгрывания дебюта.
4. Тест для определения начального уровня подготовки пользователя.
5. Итоговый тест для оценки усвоения изученного дебюта.
6. Механизм расчета эффективности обучения на основе результатов тестирования.
7. Интерактивная панель прогресса с отображением статистики пользователя.
8. История решенных задач и пройденных сценариев с возможностью повторного прохождения.
9. Серверная часть на Java Spring Boot для обработки запросов, хранения прогресса, результатов тестирования и истории пользователя в базе данных PostgreSQL.
10. Элементы онлайн-взаимодействия пользователей через игровые комнаты и WebSocket.
11. Механизм проверки корректности расчёта эффективности обучения на основе результатов теста начального уровня и итогового тестирования.

Рекомендуемые материалы

1. Oakmac — react-chessboard, React-библиотека шахматной доски.
<https://www.npmjs.com/package/react-chessboard>
2. VK — Продукты VK, документация платформы.
<https://dev.vk.com/>
3. Meta — React, библиотека для разработки пользовательских интерфейсов.
<https://react.dev>
4. VKUI — документация библиотеки интерфейсных компонентов ВКонтакте.
<https://vkcom.github.io/VKUI/>

5. chess.js — документация библиотеки для реализации шахматной логики.

<https://github.com/jhlywa/chess.js>

6. Vite — документация инструмента сборки frontend-приложений.

<https://vite.dev>

7. TypeScript — документация языка программирования.

<https://www.typescriptlang.org/docs/>

Планируемое содержание ВКР

Введение

Во введении планируется обосновать актуальность разработки веб-приложения для обучения шахматным дебютам, определить цель и задачи работы, объект и предмет исследования, а также описать практическую значимость и ожидаемый результат разработки.

Глава 1. Теоретический обзор

В первой главе планируется провести анализ предметной области обучения шахматным дебютам, рассмотреть особенности интерактивного обучения в шахматных приложениях, выполнить обзор существующих аналогов, включая Chess.com, Lichess и шахматные приложения платформы VK Games. Также будет проведён обзор технологий, используемых для реализации клиентской и серверной частей веб-приложения, сетевого взаимодействия и средств разработки.

Глава 2. Проектирование системы

Во второй главе планируется описать проектирование веб-приложения «ДебютМастер», включая общую архитектуру системы, структуру клиентской и серверной частей, взаимодействие frontend- и backend-модулей, а также проектирование базы данных.

Отдельное внимание будет уделено проектированию обучающих сценариев, механизма тестирования уровня пользователя, итогового теста, расчёта эффективности обучения и хранения пользовательского прогресса.

Глава 3. Реализация программного продукта

В третьей главе планируется описать реализацию frontend-части приложения, backend-части, интеграцию шахматной логики и шахматного движка, а также реализацию онлайн-игры.

Кроме того, будет рассмотрена реализация обучающих сценариев, теста начального уровня, итогового тестирования, панели прогресса и механизма сохранения результатов пользователя. В завершение главы будет представлено тестирование разработанного приложения, проверка корректности основных пользовательских сценариев и анализ работы механизма оценки эффективности обучения.

Заключение

В заключении планируется подвести итоги выполненной работы, оценить достижение поставленной цели, описать полученные результаты, ограничения разработанного решения и возможные направления дальнейшего развития приложения.

Форма представления материалов ВКР:

Результаты выпускной квалификационной работы будут представлены в виде пояснительной записки, включающей описание предметной области, архитектуры системы, используемых технологий и результатов разработки. Практическая часть будет представлена в виде веб-приложения, реализованного на платформе VK Games. Дополнительно будут предоставлены: – исходный программный код приложения; – схема архитектуры системы; – презентационные материалы (слайды); – демонстрация работы приложения.

Дополнительно:

Работа предполагает поэтапную реализацию функциональности: сначала разработка базовой игровой логики, затем реализация обучающих сценариев и системы отслеживания прогресса пользователя, после чего — интеграция с платформой VK и тестирование.

При наличии дополнительного времени возможно расширение функциональности приложения.

Дата выдачи задания

22.05.2026

Согласовано:

Руководитель ВКР Болдырева Елена Александровна, ФПИ и КТ, доцент (квалификационная категория "ординарный доцент"), осн.

АННОТАЦИЯ

Цель исследования

Целью выпускной квалификационной работы является повышение эффективности обучения дебютам в шахматных партиях за счёт разработки веб-приложения с интерактивными и персонализированными сценариями обучения, реализованного на платформе VK Games.

Задачи, решаемые в ВКР

Для достижения поставленной цели в работе решаются следующие задачи: анализ предметной области обучения шахматным дебютам и существующих шахматных приложений; проектирование архитектуры веб-приложения, клиентской и серверной частей, базы данных и обучающих сценариев; реализация игровой логики, локальной игры, игры против компьютерного соперника и онлайн-партий; разработка модуля обучающих сценариев, теста начального уровня, итогового теста и механизма расчёта эффективности обучения; реализация системы хранения пользовательского прогресса, истории прохождения и результатов тестирования; проведение функционального тестирования разработанного программного продукта.

Краткая характеристика полученных результатов

В результате выполнения работы разработано веб-приложение «ДебютМастер» для игры в шахматы и интерактивного обучения шахматным дебютам. Приложение включает клиентскую часть на React, TypeScript и VKUI, серверную часть на Java Spring Boot, базу данных PostgreSQL, интеграцию с шахматным движком Stockfish, онлайн-игру через игровые комнаты, обучающие сценарии, тактические задачи, тест начального уровня, итоговый тест и панель прогресса пользователя. Реализован механизм расчёта эффективности обучения на основе начального уровня подготовки пользователя и результата итогового тестирования. Проведённое функциональное тестирование подтвердило корректность работы основных пользовательских сценариев, сохранения прогресса и взаимодействия клиентской и серверной частей приложения.

Согласовано:

Руководитель ВКР Болдырева Елена Александровна, ФПИ и КТ, доцент (квалификационная категория "ординарный доцент"), осн.

24.05.2026

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	3
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	4
ВВЕДЕНИЕ.....	6
1 ТЕОРЕТИЧЕСКИЙ ОБЗОР.....	10
1.1 Актуальность разработки.....	10
1.2 Обзор аналогов.....	11
1.2.1 Платформа Chess.com.....	12
1.2.2 Платформа Lichess.....	14
1.2.3 Шахматные приложения платформы VK Games.....	16
1.2.4 Сравнительная характеристика аналогов.....	19
1.3 Обзор технологий реализации	20
1.3.1 Технологии клиентской части	20
1.3.2 Технологии серверной части	21
1.3.3 Технологии сетевого взаимодействия	21
1.3.4 Средства разработки.....	22
1.4 Алгоритм оценки эффективности обучения	22
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	26
2.1 Диаграмма вариантов использования.....	26
2.2 Диаграммы бизнес-процессов	28
2.2.1 Полная диаграмма бизнес-процесса работы приложения.....	29
2.2.2 Диаграмма бизнес-процесса прохождения обучающего сценария.....	31
2.3 Архитектура приложения	33
2.4 Проектирование клиентской части	36
2.5 Проектирование серверной части.....	39
2.6 Проектирование базы данных	42

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА	46
3.1 Реализация frontend-части.....	46
3.2 Реализация backend-части.....	54
3.3 Интеграция шахматного движка.....	59
3.4 Реализация онлайн-игры	62
3.5 Тестирование	66
ЗАКЛЮЧЕНИЕ	74
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	77

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API (Application Programming Interface) – интерфейс взаимодействия программных компонентов

CSS (Cascading Style Sheets) – язык описания внешнего вида веб-страниц

HTML (HyperText Markup Language) – язык разметки гипертекста

HTTP (HyperText Transfer Protocol) – протокол передачи данных

JSON (JavaScript Object Notation) – текстовый формат обмена данными

REST API – архитектурный подход взаимодействия клиентской и серверной частей приложения

SQL (Structured Query Language) – язык структурированных запросов

UI (User Interface) – пользовательский интерфейс

UX (User Experience) – пользовательский опыт взаимодействия с системой

WebSocket – протокол двустороннего обмена данными между клиентом и сервером в режиме реального времени

БД – база данных

FEN (Forsyth–Edwards Notation) – шахматная нотация для описания текущего состояния позиции на доске

JPA (Java Persistence API) – спецификация Java для работы с реляционными базами данных

STOMP (Simple Text Oriented Messaging Protocol) – протокол обмена сообщениями, используемый поверх WebSocket

VK Bridge – библиотека для взаимодействия веб-приложения с платформой VK Mini Apps

VKUI – библиотека интерфейсных компонентов для приложений экосистемы VK

UML (Unified Modeling Language) – унифицированный язык моделирования программных систем

ER (Entity–Relationship) – модель описания сущностей и связей базы данных

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Шахматный тренажёр – программная система для обучения и практики шахматных навыков

Шахматный движок – программный модуль для анализа шахматных позиций и генерации ходов

ДебютМастер – разрабатываемое веб-приложение для игры в шахматы и интерактивного обучения шахматным дебютам на платформе VK Games.

Тест уровня – модуль приложения, предназначенный для определения начального уровня подготовки пользователя перед прохождением обучающих сценариев.

Итоговый тест – модуль приложения, предназначенный для проверки усвоения пользователем изученного дебютного материала.

Эффективность обучения – показатель, рассчитываемый на основе начального уровня подготовки пользователя и результата итогового тестирования.

Обучающий сценарий – интерактивный сценарий изучения шахматного дебюта или тактической позиции, включающий демонстрацию ходов, подсказки и самостоятельное повторение.

Дебют – начальная стадия шахматной партии

Тактическая задача – шахматная позиция, требующая нахождения лучшего хода

Онлайн-партия – шахматная партия между пользователями через сеть Интернет

Локальная партия – режим шахматной партии, при котором оба игрока выполняют ходы в рамках одного экземпляра приложения без подключения к онлайн-комнате

Игровая комната – виртуальная игровая сессия с уникальным кодом, используемая для подключения пользователей к онлайн-партии.

История партий – сохранённый список ранее сыгранных партий пользователя

Прогресс пользователя – совокупность данных о прохождении тренировок, дебютов и сыгранных партий

Пользовательский интерфейс – совокупность визуальных элементов взаимодействия пользователя с приложением

Клиентская часть приложения – часть программного обеспечения, отвечающая за отображение интерфейса и взаимодействие с пользователем

Серверная часть приложения – часть программного обеспечения, обеспечивающая обработку данных, сетевое взаимодействие и хранение информации

FEN – текстовый формат записи шахматной позиции, включающий расположение фигур, сторону хода, возможность рокировки, взятие на проходе и дополнительные параметры позиции.

VK Games – игровая платформа экосистемы VK, используемая для размещения и запуска веб-приложения «ДебютМастер».

VK Bridge – программный интерфейс, используемый для взаимодействия клиентской части приложения с сервисами платформы VK.

WebSocket-соединение – сетевое соединение, обеспечивающее двусторонний обмен данными между клиентом и сервером в режиме реального времени.

REST API – способ организации взаимодействия клиентской и серверной частей приложения через HTTP-запросы.

ВВЕДЕНИЕ

В последние годы цифровые технологии всё активнее внедряются в образовательную сферу, обеспечивая новые возможности для организации учебного процесса, персонализации обучения и интерактивного взаимодействия пользователя с программными системами. Особенно заметно развитие веб-платформ и онлайн-сервисов, позволяющих получать доступ к образовательным материалам независимо от используемого устройства и местоположения пользователя. Одним из направлений, в котором цифровые технологии получили широкое распространение, является обучение интеллектуальным видам деятельности, включая шахматы.

Шахматы представляют собой сложную интеллектуальную игру, требующую развитого логического мышления, способности анализировать большое количество вариантов и принимать решения в условиях ограниченного времени. В процессе подготовки шахматистов важную роль играет изучение дебютов — начальной стадии партии, в рамках которой формируется стартовая позиционная структура, определяется темп развития фигур и создаются предпосылки для дальнейшей стратегии игры. Ошибки, допущенные в дебюте, способны существенно повлиять на дальнейший ход партии, поэтому качественное освоение дебютных схем является одной из ключевых задач подготовки игроков различного уровня.

Традиционные методы изучения шахматных дебютов обычно основаны на использовании специализированной литературы, разборе партий, просмотре обучающих видеоматериалов и работе с шахматными анализаторами. Несмотря на эффективность подобных подходов, они имеют ряд недостатков. Пользователь часто сталкивается с большим объёмом теоретической информации, сложностью систематизации изучаемого материала и отсутствием интерактивной практики. Кроме того, многие существующие шахматные платформы, включая Chess.com и Lichess, ориентированы преимущественно на игровой процесс, анализ партий или

справочное изучение дебютов, а не на персонализированное обучение дебютам и постепенное закрепление навыков [1, 2].

Для повышения качества обучения важно не только предоставить пользователю набор дебютных сценариев, но и учитывать его начальный уровень подготовки. В рамках разрабатываемого приложения предусматривается тестирование пользователя перед началом обучения, позволяющее определить его стартовый уровень. После прохождения обучающих сценариев пользователь выполняет итоговый тест, направленный на проверку усвоения дебютного материала. На основе результатов входного и итогового тестирования формируется показатель эффективности обучения, позволяющий оценить прогресс пользователя и результативность работы с приложением.

Дополнительной проблемой является недостаточная адаптация существующих решений под особенности современных веб-платформ и игровых экосистем. Пользователям важно получать доступ к обучающим инструментам напрямую через привычные цифровые сервисы без необходимости установки отдельного программного обеспечения или сложной настройки среды. В связи с этим особую актуальность приобретает разработка веб-приложений, интегрируемых в популярные платформы и обеспечивающих удобный пользовательский опыт.

Одной из таких платформ является VK Games — игровая экосистема, предоставляющая возможность размещения веб-приложений и онлайн-игр с поддержкой интеграции сервисов социальной сети ВКонтакте [3, 4]. Использование VK Games позволяет упростить доступ пользователей к приложению, реализовать взаимодействие с платформенными сервисами и обеспечить удобную авторизацию, хранение пользовательских данных и запуск приложения непосредственно в браузере. Интеграция шахматного веб-приложения в VK Games делает систему более доступной для широкой аудитории пользователей и позволяет объединить обучающие функции с элементами игровой платформы.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью создания современного веб-приложения для обучения шахматным дебютам, сочетающего интерактивный подход, элементы персонализации, возможность практического закрепления материала и интеграцию с игровой платформой VK Games. Разработка подобной системы позволяет повысить удобство изучения дебютных сценариев, обеспечить хранение пользовательского прогресса и предоставить инструменты для тренировки как в локальном режиме, так и в онлайн-партиях.

Объектом исследования является процесс обучения шахматным дебютам с использованием цифровых образовательных инструментов.

Предметом исследования являются методы и программные средства реализации интерактивного веб-приложения для обучения шахматным дебютам.

Целью выпускной квалификационной работы является повышение эффективности обучения дебютам в шахматных партиях за счёт использования веб-приложения с интерактивными сценариями обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих аналогов шахматных платформ;
- исследовать современные технологии разработки клиентской и серверной частей веб-приложений;
- выполнить проектирование архитектуры программной системы;
- разработать клиентскую часть приложения с использованием веб-технологий;
- разработать серверную часть приложения для обработки и хранения данных;
- реализовать интеграцию с платформой VK Games и сервисами VK Bridge;

- интегрировать шахматный движок для анализа позиций и генерации ходов;
- реализовать функционал локальной игры, онлайн-партий и обучающих сценариев;
- реализовать тест определения начального уровня подготовки пользователя и итоговый тест для проверки усвоения дебютного материала;
- реализовать механизм расчёта эффективности обучения на основе результатов тестирования;
- реализовать систему хранения пользовательского прогресса, результатов тестирования и истории партий;
- провести тестирование разработанного программного продукта.

Практическая значимость работы заключается в разработке интерактивного шахматного веб-приложения «ДебютМастер», предназначенного для обучения шахматным дебютам и интегрированного в платформу VK Games. Разработанная система позволяет пользователям изучать дебютные схемы, тренировать навыки принятия решений, проходить тестирование уровня подготовки, выполнять итоговую проверку усвоения материала, отслеживать собственный прогресс и взаимодействовать с другими игроками в рамках единой цифровой платформы.

1 ТЕОРЕТИЧЕСКИЙ ОБЗОР

1.1 Актуальность разработки

Шахматы являются интеллектуальной игрой, в которой результат партии во многом зависит не только от знания правил, но и от способности игрока анализировать позицию, выбирать оптимальный план и заранее понимать типовые игровые структуры. Одним из наиболее важных этапов шахматной партии является дебют — начальная стадия игры, в которой формируется дальнейший характер позиции, определяется развитие фигур и создаются условия для перехода в миттельшпиль.

Изучение дебютов представляет сложность для начинающих и продолжающих игроков, поскольку требует не только механического запоминания последовательности ходов, но и понимания их стратегического смысла. При изучении дебютных вариантов пользователь должен видеть, какие фигуры перемещаются, почему выполняется конкретный ход, какие задачи решаются на данном этапе и к каким позиционным последствиям это приводит. Если обучение сводится только к чтению списка ходов, например «1. e4 e5 2. Nf3 Nc6», то восприятие материала становится менее наглядным, а процесс запоминания — более трудоёмким.

На практике многие обучающие материалы по дебютам представлены в виде текстовых описаний, статей, видеоуроков или баз партий. Такие форматы полезны для теоретического изучения, однако не всегда обеспечивают активное взаимодействие пользователя с позицией. При этом для эффективного обучения важно, чтобы пользователь мог не только прочитать или посмотреть разбор, но и самостоятельно повторить изучаемую линию, сделать ход на доске, получить подсказку, вернуться к предыдущему шагу и закрепить материал без постоянного обращения к внешним источникам.

Дополнительным подтверждением актуальности выбранного направления является экспертное мнение представителя шахматного клуба Университета ИТМО. В ходе предварительного обсуждения идеи было

отмечено, что интерактивная демонстрация дебютов с визуальным перемещением фигур и возможностью повторения ходов пользователем может быть полезной, поскольку позволяет обучающемуся лучше запоминать происходящее на доске. Также было отмечено, что подобный подход отличается от простого представления дебютной теории в текстовом виде и может быть применим для пользователей, которые уже знают правила шахмат, но хотят улучшить понимание популярных дебютных схем.

Актуальность разработки также связана с необходимостью адаптации обучающих шахматных инструментов под современные веб-платформы. Размещение приложения на платформе VK Games позволяет пользователю запускать тренажёр в привычной цифровой среде без установки отдельного программного обеспечения. Такой подход повышает доступность системы и позволяет объединить обучающие функции, игровой режим, онлайн-взаимодействие и хранение пользовательского прогресса в рамках одного приложения.

Таким образом, разработка веб-приложения для обучения шахматным дебютам является актуальной задачей, поскольку существующие способы изучения дебютов не всегда обеспечивают достаточную интерактивность, наглядность и персонализацию обучения. Предлагаемое решение ориентировано на создание инструмента, в котором пользователь может изучать дебютные сценарии пошагово, получать объяснения ходов, повторять изученный материал самостоятельно и отслеживать собственный прогресс.

1.2 Обзор аналогов

В рамках исследования были рассмотрены существующие шахматные платформы и приложения, предоставляющие функции обучения дебютам, решения тактических задач и анализа партий. Основной целью анализа являлось выявление сильных и слабых сторон существующих решений, а также определение возможностей для разработки собственного веб-приложения «ДебютМастер».

При анализе особое внимание уделялось следующим критериям:

- наличие интерактивного обучения дебютам;
- наличие пошагового объяснения ходов;
- поддержка русского языка;
- наличие системы отслеживания прогресса;
- удобство пользовательского интерфейса;
- интеграция с игровой платформой VK Games.

1.2.1 Платформа Chess.com

Chess.com является одной из наиболее популярных онлайн-платформ для игры и обучения шахматам. Платформа предоставляет пользователям широкий набор возможностей, включая онлайн-партии, решение тактических задач, анализ сыгранных партий и изучение дебютов [1].

В разделе обучения дебютам пользователю предлагаются отдельные обучающие курсы по различным шахматным началам. Система содержит набор дебютов, сгруппированных по тематикам и уровням сложности [1]. Раздел обучения дебютам на платформе Chess.com представлен на рисунке 1.

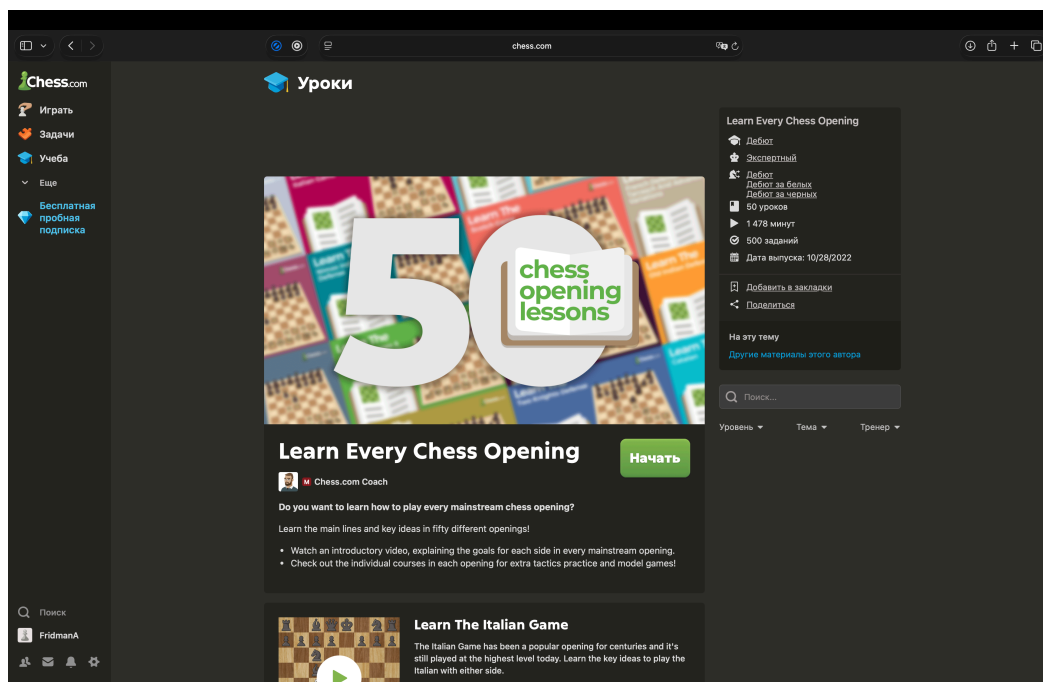


Рисунок 1 – Раздел обучения дебютам на платформе Chess.com

Платформа Chess.com является одной из наиболее популярных шахматных онлайн-платформ и предоставляет широкий набор функций для

обучения и игрового процесса. Пользователям доступны онлайн-партии, решение тактических задач, анализ сыгранных партий, а также разделы для изучения дебютов.

В разделе обучения дебютам представлены отдельные обучающие курсы по различным шахматным началам. Пользователь может изучать популярные дебюты, просматривать видеоуроки и знакомиться с основными вариантами развития фигур. Интерфейс интерактивного обучения дебюту на платформе Chess.com представлен на рисунке 2.

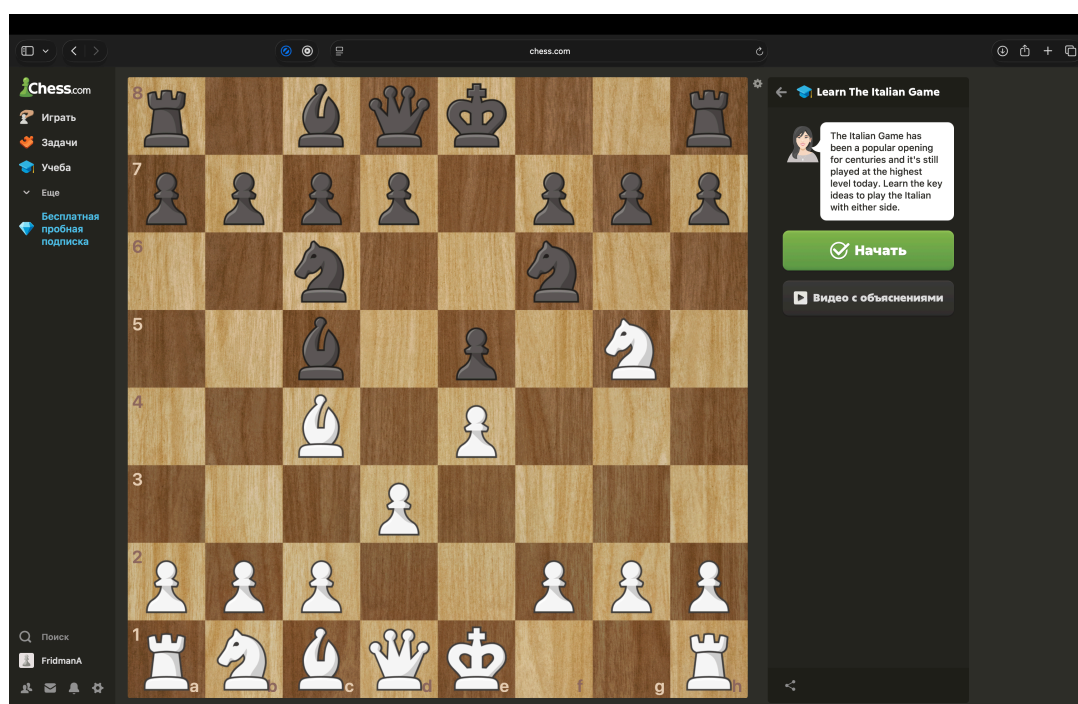


Рисунок 2 – Интерфейс интерактивного обучения дебюту на платформе Chess.com

Обучение на платформе сопровождается интерактивной шахматной доской и визуальным отображением ходов фигур. Это позволяет пользователю воспринимать дебют не только в текстовом виде, но и в формате практической демонстрации.

Несмотря на широкий набор функций, рассмотренное решение имеет ряд ограничений. Большая часть обучающих материалов и интерфейса представлена на английском языке, что может затруднять использование платформы русскоязычными пользователями. Кроме того, объяснение дебютов преимущественно реализовано в формате видеоматериалов на

английском языке без подробного текстового пошагового сопровождения ходов, что усложняет восприятие информации для начинающих пользователей.

Также платформа ориентирована преимущественно на универсальный шахматный сервис, а не на специализированное интерактивное обучение дебютам с персонализированным отслеживанием прогресса пользователя.

Таким образом, Chess.com предоставляет развитые инструменты для изучения шахматных дебютов, однако часть обучающих возможностей ограничена языковым барьером и недостаточной адаптацией под пошаговое интерактивное обучение начинающих пользователей.

1.2.2 Платформа Lichess

Lichess представляет собой бесплатную онлайн-платформу для игры в шахматы и анализа партий. Пользователям доступны онлайн-игры, решение тактических задач, анализ партий и инструменты для изучения дебютов [2].

В отличие от Chess.com, платформа ориентирована преимущественно на игровой процесс и анализ шахматных позиций. Раздел изучения дебютов реализован в виде справочной базы с вариантами продолжения ходов и статистикой популярных дебютных линий [2].

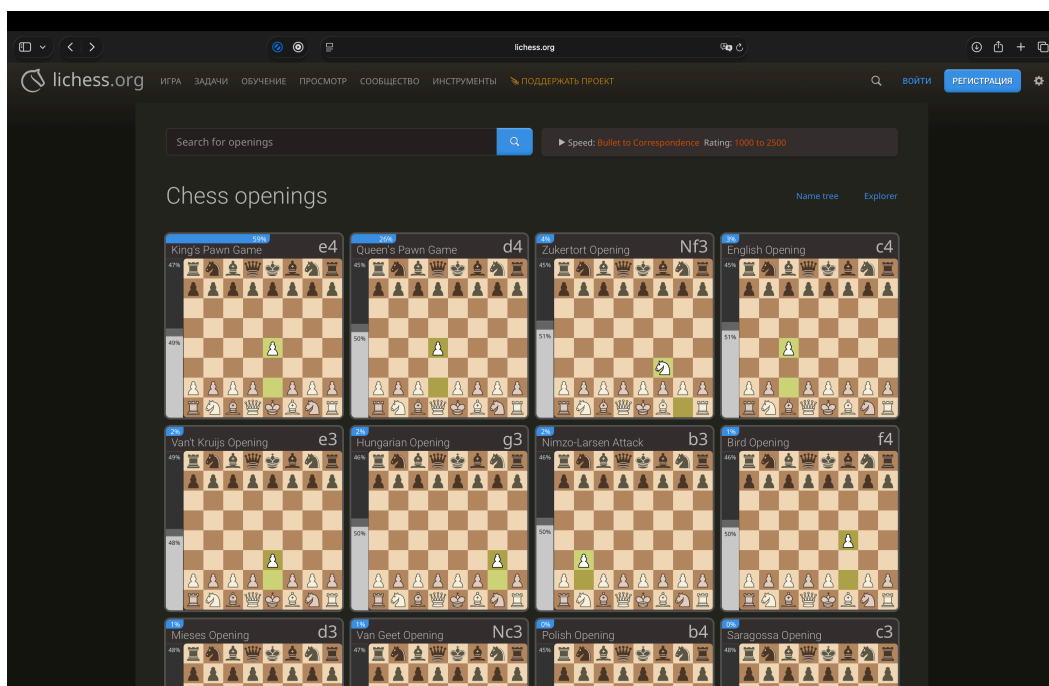


Рисунок 3 – Раздел дебютов на платформе Lichess

Как видно из рисунка 3, пользователю предоставляется список дебютов с отображением начальных ходов и статистики популярности. Интерфейс позволяет быстро переходить между различными вариантами дебютных линий и анализировать возможные продолжения партии.

При этом раздел дебютов в основном выполняет роль справочного инструмента. Полноценное интерактивное обучение с пошаговым объяснением действий пользователя на платформе реализовано ограниченно.

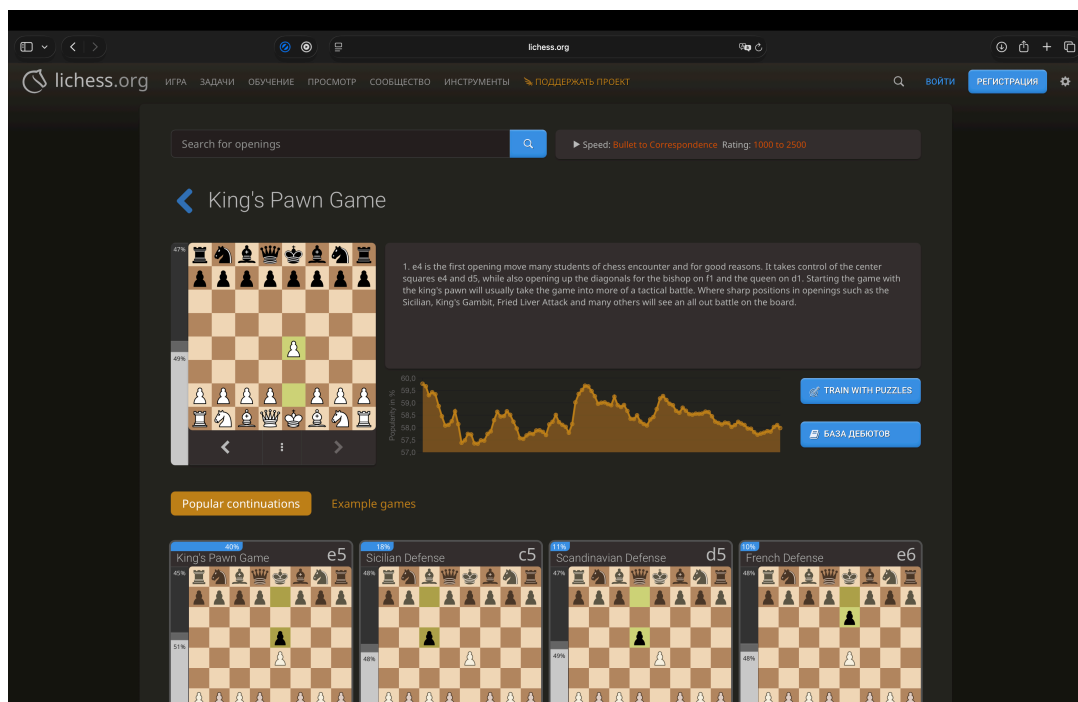


Рисунок 4 – Интерфейс просмотра дебютной позиции на платформе Lichess

На рисунке 4 представлен интерфейс просмотра дебютной позиции. Пользователю отображается шахматная доска, возможные продолжения ходов, статистическая информация и краткое описание дебюта.

Несмотря на наличие базового текстового описания, объяснение дебютов представлено преимущественно в справочном формате без полноценного интерактивного сопровождения обучения. Также значительная часть описаний и элементов интерфейса представлена на английском языке.

Кроме того, на платформе отсутствует система персонализированных обучающих сценариев и последовательного закрепления дебютов для начинающих пользователей.

Таким образом, платформа Lichess предоставляет удобные инструменты для анализа дебютов и изучения статистики шахматных позиций, однако не ориентирована на полноценное интерактивное обучение дебютам начинающих пользователей.

1.2.3 Шахматные приложения платформы VK Games

На платформе VK Games представлено большое количество шахматных приложений, ориентированных преимущественно на игровой процесс [3, 4]. Пользователям доступны онлайн-партии, игра против компьютерного соперника, а также базовые элементы статистики и соревновательного взаимодействия.

На рисунке 5 представлены шахматные приложения, размещённые на платформе VK Games.

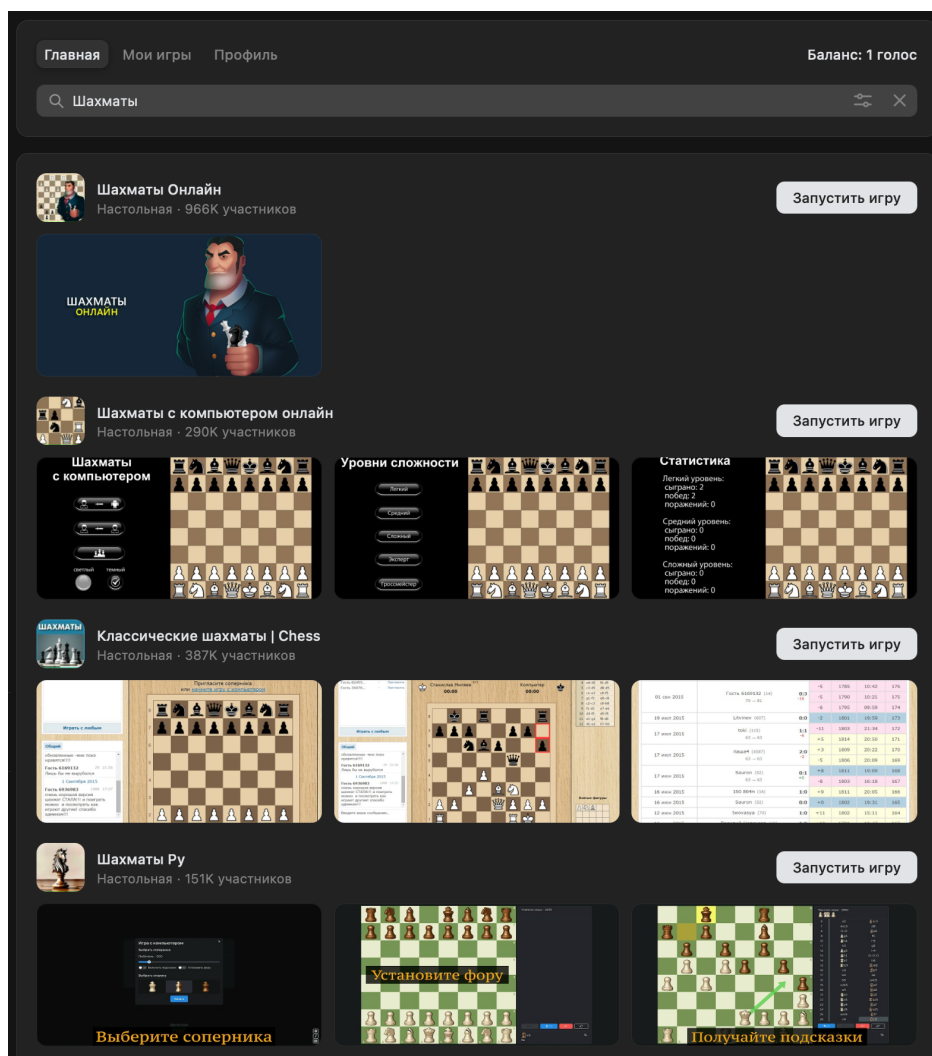


Рисунок 5 – Шахматные приложения на платформе VK Games

Как видно из рисунка 5, на платформе представлено множество шахматных приложений различных разработчиков. Основной функционал данных решений связан с проведением онлайн-партий, игрой против компьютерного соперника и соревновательным процессом.

На рисунке 6 представлен интерфейс приложения «Шахматы Онлайн». Данное приложение ориентировано преимущественно на проведение онлайн-партий между пользователями. Пользователю доступны создание игровых комнат, выбор соперников и участие в соревновательных матчах. При этом в приложении отсутствуют обучающие материалы, интерактивные сценарии изучения дебютов и пошаговое объяснение ходов.

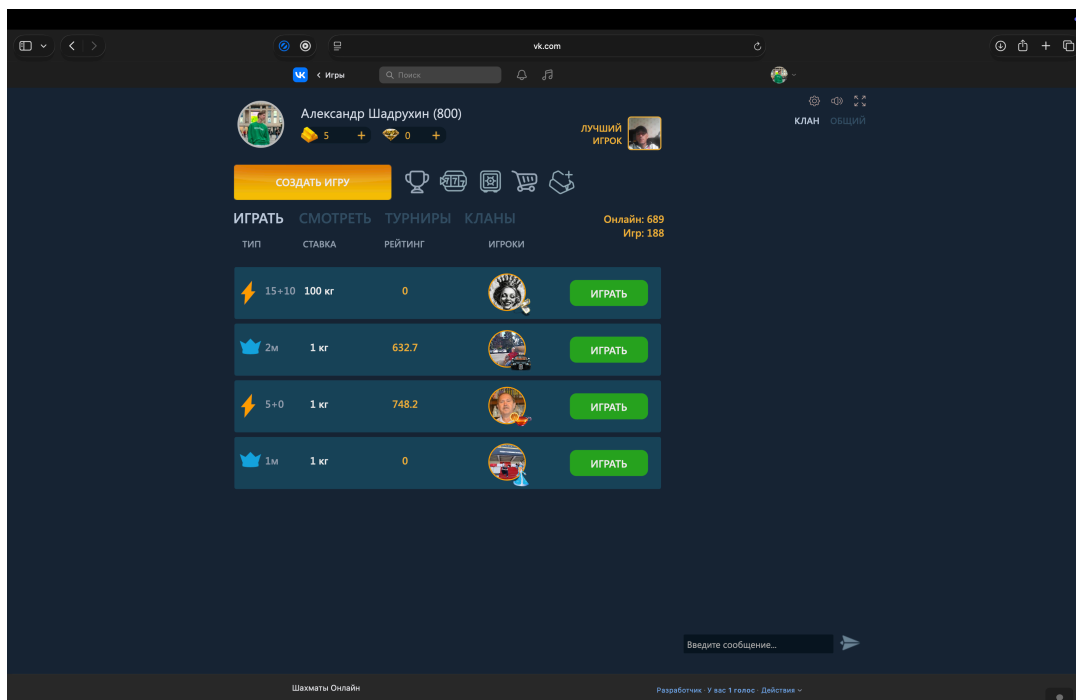


Рисунок 6 – Интерфейс приложения «Шахматы Онлайн» на платформе VK Games

На рисунке 7 представлен интерфейс приложения «Шахматы с компьютером онлайн». Пользователю доступны режимы игры против компьютерного соперника, случайного онлайн-игрока, а также локальная игра на одном устройстве. Дополнительно приложение содержит раздел статистики, в котором сохраняется информация о сыгранных партиях против компьютера.

Несмотря на наличие нескольких игровых режимов, приложение не предоставляет встроенных средств обучения шахматным дебютам. В системе отсутствуют интерактивные обучающие сценарии, объяснение логики ходов и персонализированное сопровождение пользователя в процессе обучения.



Рисунок 7 – Интерфейс приложения «Шахматы с компьютером онлайн»

Таким образом, рассмотренные шахматные приложения платформы VK Games ориентированы преимущественно на игровой процесс и соревновательное взаимодействие между пользователями. Несмотря на наличие режимов игры против компьютера и системы статистики, данные решения не предоставляют полноценных средств интерактивного изучения дебютов и обучения пользователей шахматной теории. Это подтверждает актуальность разработки веб-приложения «ДебютМастер», ориентированного на интерактивное и персонализированное обучение шахматным дебютам.

1.2.4 Сравнительная характеристика аналогов

Для определения преимуществ и недостатков существующих решений был проведён сравнительный анализ рассмотренных аналогов. В качестве критериев сравнения были выбраны наличие интерактивного обучения дебютам, пошагового объяснения ходов, русскоязычного интерфейса и интеграции с платформой VK Games. Результаты сравнительного анализа аналогов представлены в таблице 1.

Таблица 1 – Сравнительная характеристика аналогов

Критерий	Chess.com	Lichess	VK Games	ДебютМастер
Онлайн-партии	+	+	+	+
Игра против компьютера	+	+	+	+
Обучение дебютам	+	-	-	+
Пошаговое объяснение ходов	+	-	-	+
Русскоязычный интерфейс	±	±	+	+
Интеграция с VK Games	-	-	+	+
Персонализированное обучение	-	-	-	+

Примечание: «+» – функция присутствует; «-» – функция отсутствует; «±» – функция реализована частично.

На основании проведённого сравнительного анализа можно сделать вывод, что существующие решения либо ориентированы преимущественно на игровой процесс, либо предоставляют ограниченные возможности для интерактивного изучения дебютов. Разрабатываемое веб-приложение «ДебютМастер» сочетает игровые возможности, обучение дебютам и интеграцию с платформой VK Games.

1.3 Обзор технологий реализации

Для реализации веб-приложения «ДебютМастер» был выбран набор современных технологий, обеспечивающих разработку пользовательского интерфейса, серверной части приложения, обработку сетевого взаимодействия и интеграцию с платформой VK Games. Используемые технологии позволяют обеспечить стабильную работу системы, поддержку интерактивного взаимодействия пользователей и возможность дальнейшего масштабирования приложения.

1.3.1 Технологии клиентской части

Для интеграции приложения с платформой VK Games использовались библиотеки VK Bridge и VKUI [5, 6]. VK Bridge обеспечивает взаимодействие веб-приложения с API платформы VK, а VKUI предоставляет набор готовых интерфейсных компонентов, адаптированных под экосистему VK.

Для разработки клиентской части приложения использовались технологии React и TypeScript [7, 8]. React представляет собой библиотеку для создания пользовательских интерфейсов, обеспечивающую компонентный подход к разработке и эффективное обновление элементов интерфейса. Использование React позволило реализовать интерактивную шахматную доску, систему отображения ходов и пользовательские элементы управления приложением.

TypeScript использовался для повышения надёжности разработки за счёт статической типизации данных. Применение TypeScript позволяет снизить вероятность ошибок при разработке и упростить сопровождение проекта.

В качестве инструмента сборки проекта использовалась система Vite, обеспечивающая высокую скорость запуска проекта и сборки клиентской части приложения [9].

Для отображения интерактивной шахматной доски применяется библиотека react-chessboard, а для проверки ходов и работы с шахматными правилами используется библиотека chess.js [10, 11].

Для реализации игры против компьютера на клиентской стороне используется шахматный движок Stockfish [12, 13]. Использование данного движка позволяет выполнять анализ шахматной позиции и генерацию ответных ходов непосредственно в браузере пользователя без необходимости постоянного обращения к серверной части приложения.

1.3.2 Технологии серверной части

Серверная часть приложения реализована на языке программирования Java с использованием фреймворка Spring Boot [14, 15]. Данный фреймворк обеспечивает создание REST API, обработку HTTP-запросов и организацию серверной логики приложения.

Взаимодействие серверной части с базой данных реализовано с использованием технологии Spring Data JPA [16]. Для организации обмена данными в режиме реального времени используется WebSocket/STOMP-взаимодействие [17]. Для хранения пользовательских данных и информации о партиях применяется система управления базами данных PostgreSQL [18].

Дополнительно в проекте используется библиотека Lombok, позволяющая сократить объём шаблонного кода и упростить разработку серверных компонентов [19].

1.3.3 Технологии сетевого взаимодействия

Для организации взаимодействия между клиентской и серверной частями приложения используются технологии REST API и WebSocket [15, 17].

REST API применяется для передачи данных между компонентами системы, получения информации о пользователях, партиях и обучающих сценариях.

Технология WebSocket используется для реализации взаимодействия пользователей в режиме реального времени при проведении онлайн-партий. Использование WebSocket позволяет обеспечить двусторонний обмен данными между клиентом и сервером без необходимости постоянного обновления страницы [17].

1.3.4 Средства разработки

В процессе разработки приложения использовались современные средства разработки и системы управления проектом. Для разработки клиентской и серверной частей приложения использовалась интегрированная среда разработки IntelliJ IDEA.

Контроль версий проекта осуществлялся с использованием системы Git и платформы GitHub. Для тестирования и отладки пользовательского интерфейса применялись современные веб-браузеры со встроенными инструментами разработчика.

Для контейнеризации backend-части использовался Dockerfile, описывающий запуск серверного приложения в контейнере [20].

Клиентская часть приложения была развёрнута на платформе Vercel, предназначенной для публикации современных веб-приложений [21]. Серверная часть и база данных были развёрнуты на платформе Render [22].

1.4 Алгоритм оценки эффективности обучения

В рамках веб-приложения «ДебютМастер» реализован механизм оценки эффективности обучения пользователя шахматным дебютам. Данный механизм предназначен для определения результата обучения на основе двух составляющих: начального уровня подготовки пользователя и результата итогового тестирования.

Необходимость такого механизма связана с тем, что одинаковый результат итогового теста может иметь разное значение для пользователей с различным уровнем подготовки. Например, для начинающего игрока частичное успешное воспроизведение дебютной последовательности может свидетельствовать о положительном результате обучения, тогда как для более опытного пользователя такой же результат может быть недостаточным. Поэтому в приложении используется персонализированный подход к интерпретации итогового результата.

Первым этапом оценки является прохождение теста начального уровня. Пользователь отвечает на набор вопросов, связанных с базовыми знаниями шахмат, пониманием правил, шахматной нотации, принципов дебюта и опытом игры. По результатам теста пользователь относится к одной из трёх групп подготовки:

- от 0 до 3 баллов — новичок;
- от 4 до 7 баллов — средний уровень;
- от 8 до 10 баллов — уверенный уровень.

Результат теста уровня сохраняется в системе и используется при дальнейшем расчёте эффективности обучения. После изучения обучающих сценариев пользователь проходит итоговый тест, в рамках которого должен воспроизвести изученную дебютную последовательность без подсказок. Итоговый результат рассчитывается как процент правильно выполненных ходов от общего количества ходов теста.

Процент итогового теста определяется по формуле:

$$P = \frac{R}{T} \times 100,$$

где

P — процент прохождения итогового теста;

R — количество правильно выполненных ходов;

T — общее количество ходов в итоговом тесте.

После этого результат корректируется с учётом уровня подготовки пользователя. Для этого используется коэффициент уровня. Чем выше начальный уровень пользователя, тем строже оценивается результат итогового теста. В приложении используются следующие коэффициенты:

- для новичка — 1,00;
- для пользователя среднего уровня — 0,85;
- для уверенного пользователя — 0,70.

Итоговая эффективность обучения рассчитывается по формуле:

$$E = P \times K,$$

где

Е — эффективность обучения;

Р — процент прохождения итогового теста;

К — коэффициент, соответствующий уровню подготовки пользователя.

Например, если пользователь среднего уровня выполнил итоговый тест на 80%, то эффективность обучения составит:

$$E = 80 \times 0,85 = 68\%.$$

Если такой же результат показал пользователь с уровнем «новичок», его эффективность составит:

$$E = 80 \times 1,00 = 80\%.$$

Если пользователь с уровнем «уверенный» выполнил итоговый тест на 80%, эффективность составит:

$$E = 80 \times 0,70 = 56\%.$$

Таким образом, алгоритм учитывает не только абсолютный результат итогового теста, но и начальный уровень подготовки пользователя. Это позволяет сделать оценку более персонализированной и использовать её как элемент анализа прогресса обучения.

Для удобства отображения результата в интерфейсе числовое значение эффективности дополнительно преобразуется в текстовую интерпретацию:

- от 0 до 39% — низкая эффективность;
- от 40 до 69% — средняя эффективность;

– от 70 до 100% — высокая эффективность.

Рассчитанное значение эффективности сохраняется в базе данных вместе с результатами теста уровня и итогового теста. Эти данные отображаются в панели прогресса пользователя и позволяют оценить, насколько успешно пользователь усвоил изучаемый дебютный материал.

Таким образом, алгоритм оценки эффективности обучения является одним из элементов персонализации веб-приложения «ДебютМастер». Он связывает начальное тестирование, итоговую проверку знаний и отображение прогресса пользователя, что позволяет рассматривать приложение не только как шахматный тренажёр, но и как систему контроля результатов обучения.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Диаграмма вариантов использования

Диаграмма вариантов использования была выбрана в качестве одного из основных средств проектирования, поскольку она позволяет описать систему с точки зрения доступных пользователю функций, не углубляясь на данном этапе в детали программной реализации. Такой подход удобен для предварительного анализа требований к приложению и определения границ разрабатываемой системы.

В рамках проектируемого веб-приложения пользователь взаимодействует не с отдельными программными модулями напрямую, а с функциональными разделами интерфейса. Поэтому варианты использования были сгруппированы по смысловым блокам: запуск и навигация, игровой режим, обучающий режим и прогресс пользователя. Такое разделение позволяет наглядно показать, какие действия относятся к игровому процессу, какие — к обучению шахматным дебютам, а какие — к просмотру результатов и истории прохождения.

Блок «Запуск и навигация» отражает начальные действия пользователя при работе с приложением. К ним относятся открытие приложения, получение идентификатора пользователя через платформу VK Games и выбор основного раздела системы. Данный блок является общим для всех последующих сценариев, так как пользователь сначала попадает на главный экран приложения, а затем выбирает необходимый режим работы.

Блок «Игровой режим» включает сценарии, связанные с непосредственным проведением шахматных партий. Пользователь может начать локальную партию на одном устройстве, выбрать игру против компьютерного соперника или перейти к онлайн-партии. Для онлайн-режима предусмотрены создание игровой комнаты, подключение по коду, продолжение ранее начатой партии и выход из комнаты. В режиме игры против

компьютера используется внешний шахматный движок Stockfish, который формирует ответный ход компьютерного соперника.

Блок «Обучающий режим» является ключевым для темы выпускной квалификационной работы, поскольку именно он реализует функциональность обучения шахматным дебютам. В данном блоке пользователь может пройти тест начального уровня, выбрать тип обучения, открыть список дебютов или тактических задач, просмотреть демонстрацию дебюта, запустить автоматический показ, повторить дебют самостоятельно, получить подсказку и пройти итоговый тест. Отдельно выделены действия по проверке хода, сохранению попытки и сохранению прогресса сценария, так как эти операции используются в нескольких обучающих режимах.

Блок «Прогресс пользователя» предназначен для отображения результатов обучения и ранее выполненных действий. В нём пользователь может просмотреть общую панель прогресса, результаты тестирования, историю прохождения сценариев, открыть сохранённую позицию, продолжить незавершённый сценарий, удалить запись из истории и обновить данные. Наличие данного блока позволяет связать обучающие сценарии с механизмом анализа результатов и расчёта эффективности обучения.

Связи между вариантами использования на диаграмме представлены отношениями `include` и `extend`. Отношение `include` используется в тех случаях, когда один сценарий обязательно включает выполнение другого действия, например прохождение итогового теста включает проверку ходов и сохранение результата. Отношение `extend` применяется для дополнительных или необязательных действий, например получение подсказки, запуск автоматического показа дебюта или удаление записи из истории.

Таким образом, построенная диаграмма позволяет определить основные функциональные возможности веб-приложения «ДебютМастер», зафиксировать границы системы и показать взаимодействие пользователя с игровыми, обучающими и аналитическими возможностями приложения.

Диаграмма вариантов использования веб-приложения «ДебютМастер» представлена на рисунке 8.

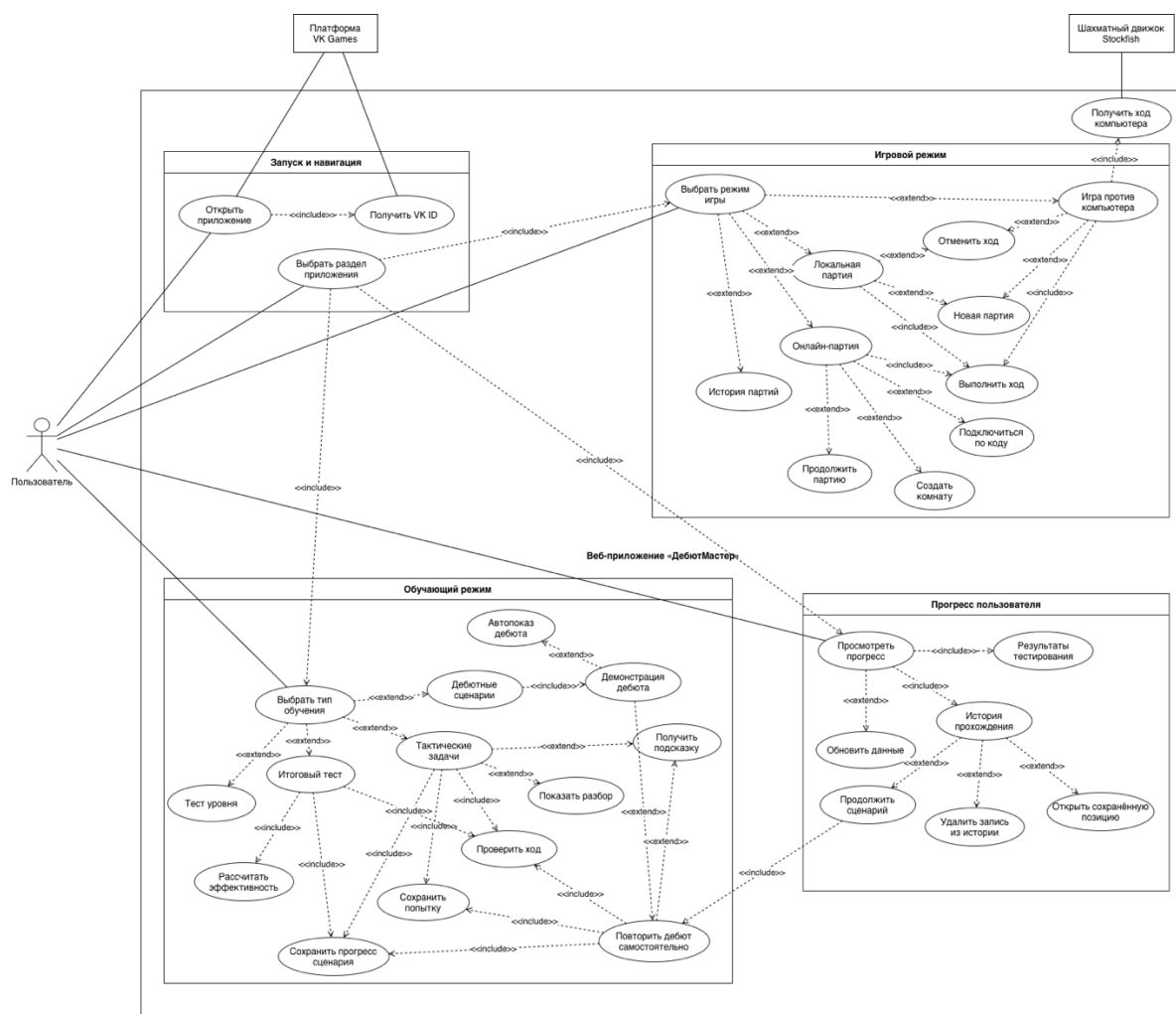


Рисунок 8 – Диаграмма вариантов использования веб-приложения «ДебютМастер»

На рисунке 8 показано, что центральным участником работы системы является пользователь. Он взаимодействует с приложением через основные разделы: игру, обучение и прогресс. Внешними участниками являются платформа VK Games, используемая для запуска приложения и получения идентификатора пользователя, а также шахматный движок Stockfish, применяемый в режиме игры против компьютера.

2.2 Диаграммы бизнес-процессов

После определения вариантов использования веб-приложения необходимо рассмотреть внутреннюю логику выполнения основных

процессов системы. Для этого были разработаны диаграммы бизнес-процессов, отражающие последовательность действий пользователя, клиентской части приложения, серверной части, базы данных и внешних компонентов.

Диаграммы бизнес-процессов позволяют наглядно представить порядок выполнения операций в системе: запуск приложения, выбор основного раздела, переход к игровому или обучающему режиму, обработку действий пользователя, сохранение данных и отображение результатов. Такой подход используется для уточнения логики взаимодействия между основными частями приложения до перехода к проектированию архитектуры и программной реализации.

В рамках проектирования были построены две диаграммы. Первая диаграмма отражает общий бизнес-процесс работы веб-приложения «ДебютМастер» и показывает основные сценарии: запуск через платформу VK Games, выбор раздела, запуск игрового режима, переход к обучению и просмотр прогресса. Вторая диаграмма является декомпозицией обучающего процесса и подробно описывает прохождение теста уровня, дебютного сценария, тактической задачи и итогового теста.

2.2.1 Полная диаграмма бизнес-процесса работы приложения

Полная диаграмма бизнес-процесса работы веб-приложения «ДебютМастер» отражает общий сценарий взаимодействия пользователя с системой. На диаграмме представлены основные участники процесса: пользователь, платформа VK Games, клиентская часть приложения, серверная часть приложения, база данных и шахматный движок Stockfish.

Работа приложения начинается с открытия веб-приложения через платформу VK Games. Платформа передаёт данные запуска приложения и предоставляет идентификатор пользователя VK. После этого клиентская часть инициализирует интерфейс, а серверная часть получает или создаёт профиль пользователя. Данные пользователя сохраняются в базе данных и используются при дальнейшей работе с игровыми и обучающими функциями.

После загрузки главного экрана пользователь выбирает один из основных разделов приложения: игровой режим, обучающий режим или раздел прогресса. При выборе игрового режима пользователю отображается меню доступных вариантов игры. Он может начать локальную партию, выбрать игру против компьютерного соперника или перейти к онлайн-режиму. В режиме игры против компьютера клиентская часть обращается к шахматному движку Stockfish, который генерирует ответный ход. В онлайн-режиме серверная часть создаёт, подключает или восстанавливает игровую комнату, а база данных хранит информацию о комнатах и партиях.

При выборе обучающего режима пользователь переходит к разделу обучения, где может пройти тест уровня, выбрать дебютный сценарий, тактическую задачу или итоговый тест. Клиентская часть отображает соответствующий сценарий, обрабатывает ход пользователя на шахматной доске, проверяет ответы и передаёт данные о прохождении на серверную часть. Сервер сохраняет попытки, ошибки, текущий прогресс сценария, результаты тестов и показатели эффективности обучения.

Раздел прогресса предназначен для просмотра сохранённых результатов пользователя. При открытии данного раздела клиентская часть запрашивает данные прогресса, серверная часть получает историю прохождения и результаты тестирования, а база данных возвращает сохранённые данные пользователя. На основе полученной информации в интерфейсе отображается панель прогресса, история прохождения и результаты оценки эффективности обучения.

Полная диаграмма бизнес-процесса работы веб-приложения «ДебютМастер» представлена на рисунке 9.

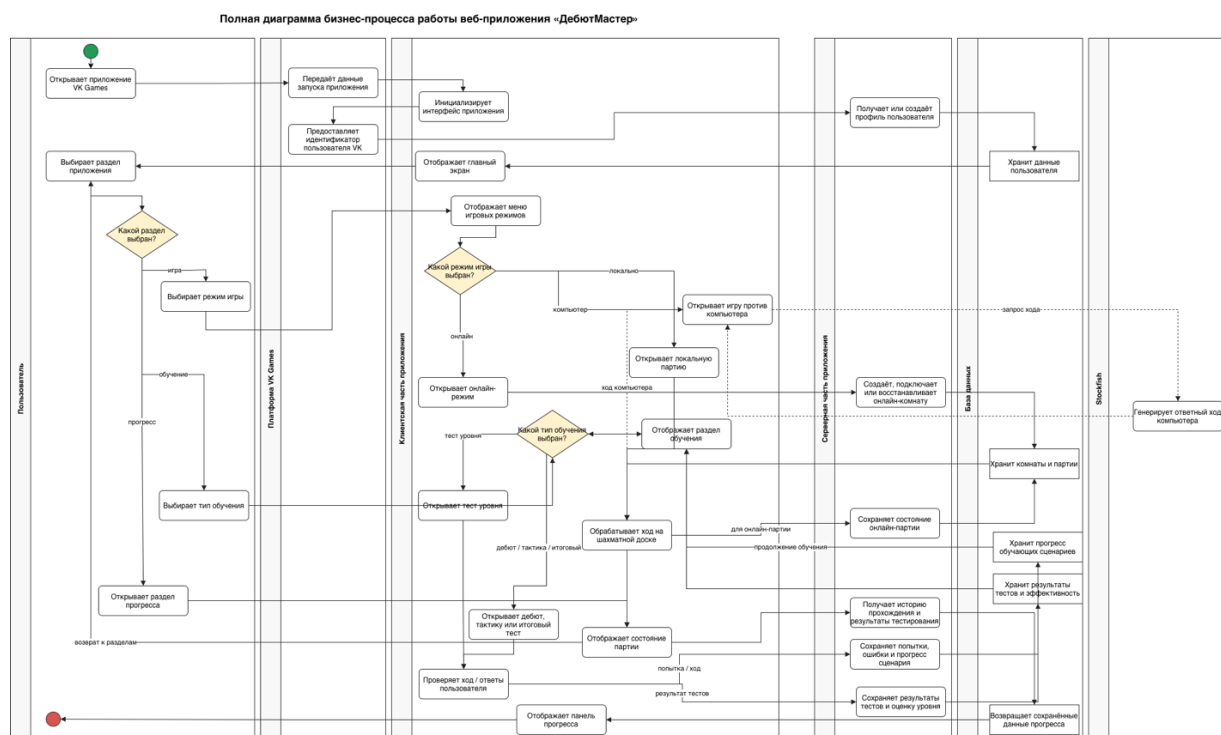


Рисунок 9 – Полная диаграмма бизнес-процесса работы веб-приложения «ДебютМастер»

Таким образом, полная диаграмма бизнес-процесса показывает общий порядок работы приложения и связывает между собой основные функциональные блоки системы: запуск через VK Games, игровой режим, обучающий режим, работу с прогрессом, серверную обработку данных и хранение информации в базе данных.

2.2.2 Диаграмма бизнес-процесса прохождения обучающего сценария

Поскольку основной целью веб-приложения «ДебютМастер» является повышение эффективности обучения шахматным дебютам, отдельного рассмотрения требует бизнес-процесс прохождения обучающего сценария. Данная диаграмма является декомпозицией обучающего режима и подробнее раскрывает действия пользователя, клиентской части, серверной части и базы данных при выполнении учебных сценариев.

Процесс начинается с открытия пользователем раздела обучения. Клиентская часть приложения отображает доступные типы обучения, после чего пользователь выбирает один из вариантов: тест уровня, дебютный

сценарий, тактическую задачу или итоговый тест. В зависимости от выбранного режима система отображает соответствующий интерфейс и запускает дальнейший сценарий работы.

При прохождении теста уровня пользователь отвечает на вопросы, после чего клиентская часть подсчитывает количество положительных ответов и определяет уровень подготовки пользователя. Серверная часть сохраняет результат теста уровня, а база данных обновляет данные оценки обучения. Данный результат используется в дальнейшем при расчёте эффективности обучения.

При прохождении дебютного сценария или тактической задачи пользователь выполняет ход на шахматной доске. Клиентская часть проверяет корректность хода и определяет дальнейшее действие. Если ход выполнен правильно, система переходит к следующему шагу сценария, фиксирует успешную попытку и сохраняет текущий прогресс. Если ход ошибочный, пользователю отображается сообщение об ошибке, а серверная часть сохраняет ошибочную попытку. При необходимости пользователь может запросить подсказку или просмотреть разбор решения.

Если сценарий завершён, серверная часть сохраняет итоговое состояние прохождения, а база данных записывает статус завершения и итоговую позицию. В случае прохождения итогового теста дополнительно сохраняется результат тестирования и рассчитывается эффективность обучения. После этого пользователь может перейти в панель прогресса, где отображаются результаты тестирования, история прохождения и рассчитанный показатель эффективности.

Диаграмма бизнес-процесса прохождения обучающего сценария представлена на рисунке 10.

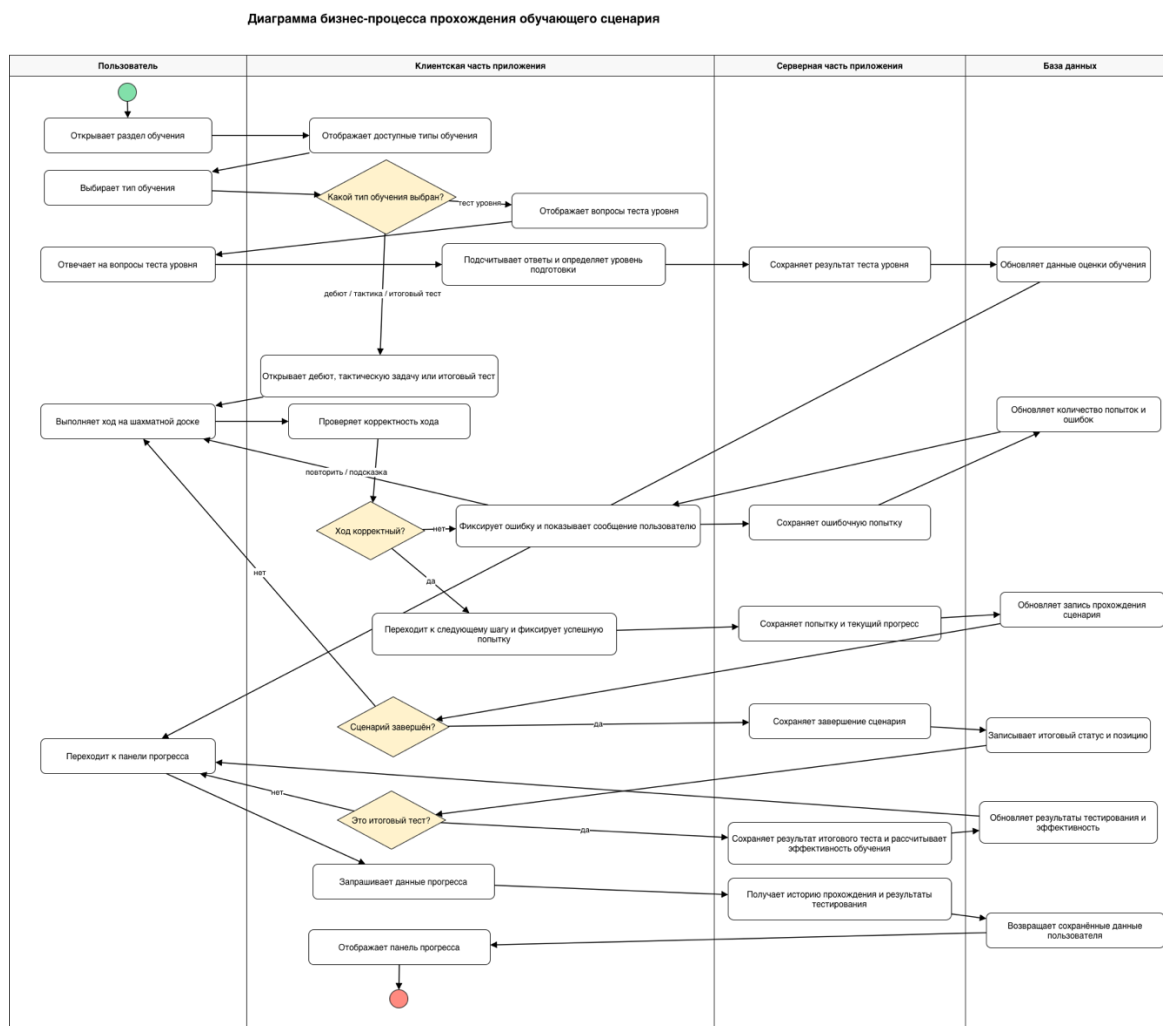


Рисунок 10 – Диаграмма бизнес-процесса прохождения обучающего сценария

Представленная диаграмма показывает, что обучающий процесс в приложении построен не только на отображении шахматных позиций, но и на постоянной фиксации действий пользователя. Система сохраняет попытки, ошибки, текущий шаг сценария, результаты тестов и итоговые показатели обучения. Это позволяет пользователю возвращаться к незавершённым сценариям, повторно проходить сложные позиции и отслеживать собственный прогресс.

2.3 Архитектура приложения

После определения вариантов использования и бизнес-процессов была спроектирована архитектура веб-приложения «ДебютМастер». Архитектура приложения построена по клиент-серверному принципу и включает

клиентскую часть, серверную часть, базу данных, внешнюю платформу VK Games и шахматный движок Stockfish.

Клиентская часть отвечает за отображение пользовательского интерфейса, обработку действий пользователя, работу шахматной доски, запуск игровых и обучающих сценариев, а также взаимодействие с серверной частью через REST API и WebSocket. Серверная часть предназначена для обработки запросов клиента, управления онлайн-комнатами, партиями, прогрессом пользователя и результатами тестирования. База данных используется для хранения пользовательских данных, информации об игровых комнатах, партиях, обучающих сценариях, прогрессе и результатах тестов.

Отдельную роль в архитектуре занимает платформа VK Games / VK Mini Apps. Пользователь запускает приложение через платформу VK, после чего с помощью VK Bridge выполняется получение идентификатора пользователя и передача данных запуска в клиентскую часть приложения. Это позволяет связать действия пользователя в приложении с его профилем и использовать полученный идентификатор при сохранении прогресса, партий и результатов обучения.

Архитектура веб-приложения «ДебютМастер» представлена на рисунке 11.

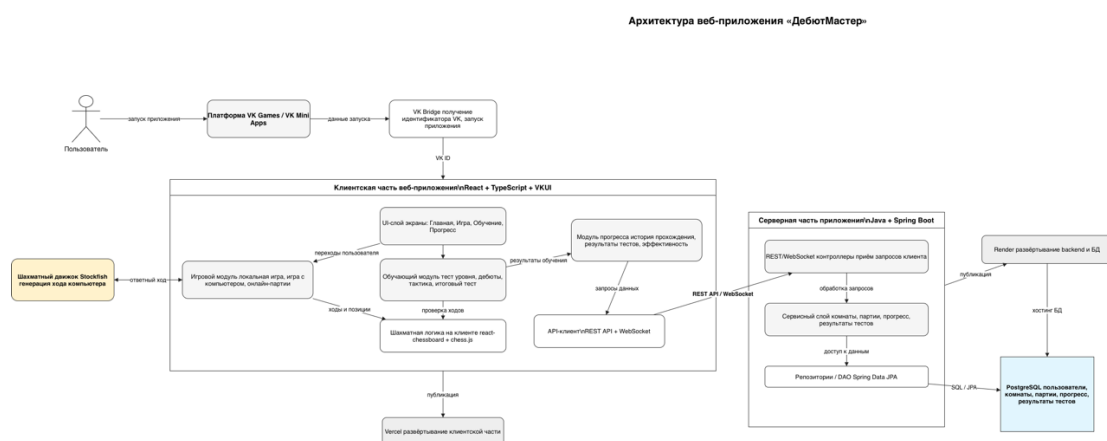


Рисунок 11 – Архитектура веб-приложения «ДебютМастер»

На рисунке 11 показано, что пользователь взаимодействует с приложением через платформу VK Games. Платформа передаёт данные запуска приложения, а VK Bridge используется для получения идентификатора

пользователя VK. После этого клиентская часть инициализирует интерфейс и отображает основные разделы приложения: главную страницу, игровой режим, обучающий режим и панель прогресса.

Клиентская часть веб-приложения реализуется с использованием React, TypeScript и VKUI [6-8]. Внутри неё выделены несколько функциональных модулей. UI-слой отвечает за отображение экранов приложения и навигацию между ними. Игровой модуль обеспечивает работу локальной партии, игры против компьютера и онлайн-партий. Обучающий модуль реализует тест начального уровня, дебютные сценарии, тактические задачи и итоговый тест. Модуль прогресса отвечает за отображение истории прохождения, результатов тестирования и рассчитанной эффективности обучения.

Шахматная логика на клиентской стороне реализуется с использованием библиотек `react-chessboard` и `chess.js` [10, 11]. Библиотека `react-chessboard` применяется для отображения интерактивной шахматной доски, а `chess.js` используется для обработки шахматных правил, проверки ходов и хранения состояния позиции. Для режима игры против компьютера дополнительно используется шахматный движок Stockfish, который генерирует ответный ход компьютерного соперника [12, 13].

Взаимодействие клиентской и серверной частей осуществляется через REST API и WebSocket [15, 17]. REST API используется для операций получения и сохранения данных, связанных с пользователем, прогрессом, результатами тестов и историей прохождения. WebSocket применяется для онлайн-взаимодействия в игровых комнатах, где требуется оперативная передача ходов и обновление состояния партии между пользователями.

Серверная часть приложения реализуется на Java с использованием Spring Boot [14, 15]. В её состав входят REST/WebSocket-контроллеры, сервисный слой и репозитории. Контроллеры принимают запросы от клиентской части, сервисный слой реализует основную бизнес-логику, а репозитории обеспечивают доступ к данным через Spring Data JPA. Серверная часть обрабатывает создание и подключение к онлайн-комнатам, сохранение

партий, фиксацию прогресса обучающих сценариев, сохранение результатов теста уровня и итогового теста.

Для хранения данных используется PostgreSQL [18]. В базе данных сохраняются сведения о пользователях, игровых комнатах, партиях, прогрессе обучающих сценариев и результатах тестирования. Хранение этих данных необходимо для восстановления состояния онлайн-партий, продолжения незавершённых сценариев, отображения истории прохождения и расчёта эффективности обучения.

Развёртывание системы выполняется с использованием облачных платформ. Клиентская часть размещается на Vercel, что обеспечивает доступ к веб-приложению через браузер. Серверная часть и база данных развёртываются на Render [21, 22]. Такое разделение позволяет независимо размещать клиентскую и серверную части приложения, а также обеспечить удалённое хранение данных и сетевое взаимодействие пользователей.

2.4 Проектирование клиентской части

Клиентская часть веб-приложения «ДебютМастер» отвечает за отображение пользовательского интерфейса, обработку действий пользователя, работу шахматной доски, навигацию между разделами приложения и взаимодействие с серверной частью. Поскольку приложение размещается на платформе VK Games, клиентская часть также должна поддерживать запуск внутри среды VK Mini Apps и получение данных пользователя через VK Bridge [4, 5].

При проектировании клиентской части был выбран компонентный подход. Интерфейс приложения разделён на отдельные экраны и переиспользуемые компоненты. Такой подход позволяет упростить разработку, изолировать логику отдельных модулей и обеспечить возможность дальнейшего расширения функциональности приложения.

Клиентская часть включает несколько основных экранов: главный экран, экран игрового режима, экран обучающих сценариев и экран прогресса

пользователя. Каждый экран отвечает за отдельный пользовательский сценарий, а переходы между ними выполняются с использованием клиентской маршрутизации.

Структура клиентской части веб-приложения «ДебютМастер» представлена на рисунке 12.

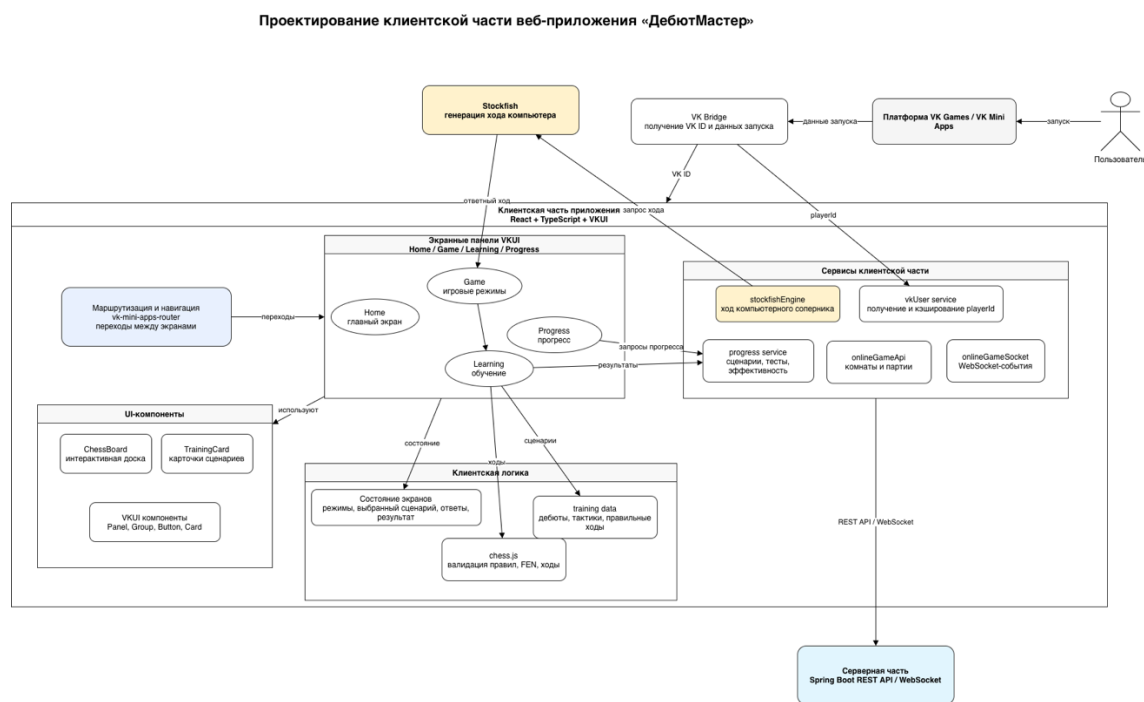


Рисунок 12 – Структура клиентской части веб-приложения «ДебютМастер»

На рисунке 12 представлена структура клиентской части приложения. Центральным элементом клиентской части является React-приложение, реализованное с использованием TypeScript и библиотеки интерфейсных компонентов VKUI [6-8]. Использование React обеспечивает компонентную организацию интерфейса, TypeScript позволяет повысить надёжность кода за счёт статической типизации, а VKUI обеспечивает визуальное соответствие приложения экосистеме VK.

Взаимодействие пользователя с приложением начинается с платформы VK Games / VK Mini Apps. После запуска приложения VK Bridge используется для получения идентификатора пользователя VK и передачи данных запуска в клиентскую часть. Полученный идентификатор применяется при обращении к серверной части для загрузки и сохранения данных пользователя, прогресса обучения и информации об онлайн-партиях.

Навигационный модуль отвечает за переходы между основными экранами приложения. В приложении предусмотрены следующие основные разделы: главный экран, игровой режим, обучающий режим и панель прогресса. Главный экран используется для выбора основного направления работы приложения. Экран игры предоставляет пользователю доступ к локальной партии, игре против компьютера и онлайн-режиму. Экран обучения содержит тест уровня, список дебютов, тактические задачи и итоговый тест. Экран прогресса отображает результаты прохождения сценариев, историю обучения и показатели эффективности.

Отдельную роль в клиентской части занимает компонент шахматной доски. Он используется как в игровом режиме, так и в обучающих сценариях. Для отображения доски применяется библиотека `react-chessboard`, а для обработки шахматных правил, проверки допустимости ходов и хранения состояния партии используется библиотека `chess.js` [10, 11]. Такое разделение позволяет отделить визуальное представление доски от логики шахматной партии.

Игровой модуль клиентской части отвечает за работу локальной партии, игры против компьютера и онлайн-партии. В локальном режиме все ходы выполняются в рамках одного экземпляра приложения. В режиме игры против компьютера клиентская часть обращается к шахматному движку `Stockfish`, который формирует ответный ход. В онлайн-режиме клиентская часть взаимодействует с сервером через `WebSocket`, получая обновления состояния партии и передавая выполненные ходы.

Обучающий модуль реализует основные функции, связанные с изучением шахматных дебютов. Он включает тест начального уровня, дебютные сценарии, тактические задачи, систему подсказок, демонстрацию правильного разыгрывания дебюта, самостоятельное повторение изученного материала и итоговый тест. В процессе выполнения сценариев клиентская часть проверяет действия пользователя, отображает результат хода, подсказки и разбор решения.

Модуль прогресса отвечает за отображение результатов обучения. Он получает данные от серверной части и показывает пользователю количество завершённых сценариев, историю прохождения, результаты теста уровня, результаты итогового теста и рассчитанную эффективность обучения. Также в данном модуле предусмотрена возможность открыть сохранённую позицию, продолжить незавершённый сценарий или удалить запись из истории.

Для взаимодействия с серверной частью в клиентском приложении выделен слой сервисов. В него входят функции для работы с REST API, WebSocket-соединением, онлайн-комнатами, прогрессом пользователя и результатами тестирования. Наличие отдельного сервисного слоя позволяет отделить сетевое взаимодействие от компонентов интерфейса и упростить сопровождение кода.

Таким образом, клиентская часть приложения спроектирована как набор взаимосвязанных экранов, компонентов и сервисов. Такая структура обеспечивает разделение ответственности между модулями интерфейса, игровой логикой, обучающими сценариями, прогрессом пользователя и сетевым взаимодействием с серверной частью.

2.5 Проектирование серверной части

Серверная часть веб-приложения «ДебютМастер» предназначена для обработки запросов клиентской части, управления онлайн-партиями, хранения прогресса пользователя и сохранения результатов обучающих сценариев. При проектировании серверной части был выбран слоистый подход, при котором ответственность распределяется между контроллерами, сервисами, репозиториями и доменными сущностями.

Серверная часть реализуется на языке Java с использованием фреймворка Spring Boot. Для обработки HTTP-запросов применяется REST API, для организации онлайн-взаимодействия между игроками используется WebSocket, а для работы с базой данных применяется Spring Data JPA [14-17]. Хранение данных осуществляется в PostgreSQL [18].

Основная логика серверной части разделена на несколько функциональных направлений. Первое направление связано с игровыми комнатами и онлайн-партиями. Второе направление отвечает за сохранение прогресса прохождения обучающих сценариев. Третье направление связано с хранением результатов теста уровня, итогового теста и показателя эффективности обучения.

Структура серверной части веб-приложения «ДебютМастер» представлена на рисунке 13.

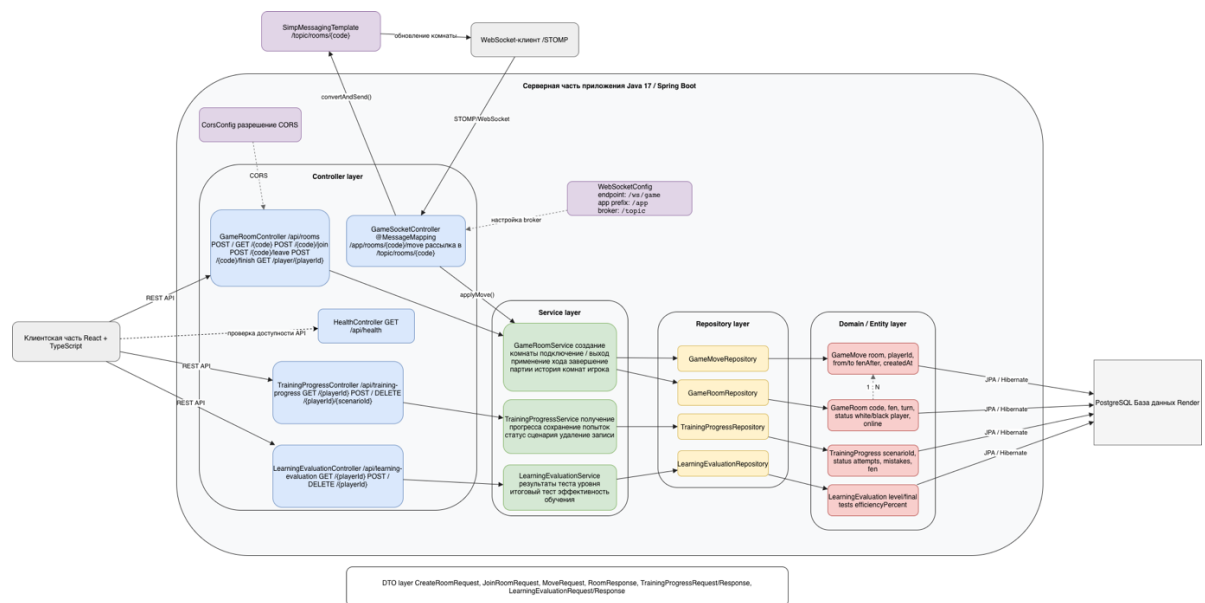


Рисунок 13 – Структура серверной части веб-приложения «ДебютМастер»

На рисунке 13 показана структура серверной части приложения. Взаимодействие клиентской части с сервером осуществляется двумя способами: через REST API и через WebSocket. REST API используется для создания и получения игровых комнат, подключения и выхода из комнат, завершения партий, получения истории партий, сохранения прогресса обучения и результатов тестирования. WebSocket используется для передачи ходов в онлайн-партиях и рассылки обновлённого состояния комнаты подключённым клиентам.

В верхней части диаграммы показан WebSocket-контур приложения. Конфигурация WebSocket задаётся классом `WebSocketConfig`, в котором определяется endpoint `/ws/game`, префикс входящих сообщений `/app` и брокер

сообщений `/topic` [17]. Клиент отправляет ход игрока по каналу `/app/rooms/{code}/move`, после чего `GameSocketController` обрабатывает сообщение, передаёт данные в `GameRoomService` и выполняет рассылку обновлённого состояния комнаты через `SimpMessagingTemplate` в канал `/topic/rooms/{code}`.

Слой контроллеров отвечает за приём запросов от клиентской части. `GameRoomController` обрабатывает REST-запросы, связанные с игровыми комнатами: создание комнаты, получение данных комнаты, подключение пользователя, выход из комнаты, завершение партии и получение списка комнат пользователя. `TrainingProgressController` отвечает за получение, сохранение и удаление записей прогресса прохождения обучающих сценариев. `LearningEvaluationController` используется для работы с результатами тестирования и оценкой эффективности обучения. `HealthController` предоставляет технический endpoint `/api/health`, предназначенный для проверки доступности серверной части.

Сервисный слой содержит основную бизнес-логику серверной части. `GameRoomService` отвечает за создание онлайн-комнат, подключение игроков, выход из комнаты, применение хода, завершение партии и получение истории комнат игрока. `TrainingProgressService` реализует получение и сохранение прогресса пользователя, включая количество попыток, ошибок, текущий шаг сценария и статус прохождения. `LearningEvaluationService` отвечает за получение и сохранение результатов теста уровня, итогового теста и рассчитанной эффективности обучения.

Слой репозиторий обеспечивает доступ к данным с использованием Spring Data JPA. `GameRoomRepository` используется для работы с игровыми комнатами, `GameMoveRepository` — для сохранения ходов в онлайн-партиях, `TrainingProgressRepository` — для хранения прогресса обучающих сценариев, `LearningEvaluationRepository` — для хранения результатов тестирования и оценки эффективности обучения.

Доменный слой представлен JPA-сущностями. GameRoom описывает игровую комнату и содержит код комнаты, текущую позицию в формате FEN, очередь хода, статус партии, идентификаторы игроков, информацию о победителе и онлайн-статусы игроков. GameMove хранит отдельные ходы онлайн-партии, включая игрока, начальную и конечную клетки, превращение фигуры и позицию после хода. TrainingProgress описывает прогресс прохождения обучающего сценария: идентификатор пользователя, сценарий, тип сценария, статус, количество попыток, ошибок, текущий шаг и сохранённые позиции. LearningEvaluation хранит результаты теста уровня, итогового теста и показатель эффективности обучения.

Для передачи данных между клиентской и серверной частями используются DTO-объекты. Они применяются при создании комнаты, подключении пользователя, передаче хода, завершении партии, сохранении прогресса и результатов тестирования. Использование DTO позволяет отделить внутреннюю структуру доменных сущностей от формата данных, передаваемых через API.

Таким образом, серверная часть приложения спроектирована по слоистой архитектуре. Контроллеры принимают запросы от клиентской части, сервисы выполняют бизнес-логику, репозитории обеспечивают доступ к данным, а доменные сущности описывают структуру сохраняемой информации. Такое разделение упрощает сопровождение проекта, делает структуру серверной части более понятной и позволяет независимо развивать отдельные функциональные модули приложения.

2.6 Проектирование базы данных

Для хранения данных веб-приложения «ДебютМастер» используется реляционная база данных PostgreSQL [18]. База данных предназначена для сохранения информации об онлайн-партиях, ходах игроков, прогрессе прохождения обучающих сценариев, результатах тестирования и рассчитанной эффективности обучения.

При проектировании базы данных учитывались основные пользовательские сценарии приложения: создание онлайн-комнаты, подключение пользователя к партии, выполнение ходов, сохранение состояния партии, прохождение обучающих сценариев, фиксация попыток и ошибок, прохождение теста начального уровня и итогового теста. На основании этих сценариев были выделены четыре основные таблицы: `game_rooms`, `game_moves`, `training_progress` и `learning_evaluation`.

ER-диаграмма базы данных веб-приложения «ДебютМастер» представлена на рисунке 14.

ER-диаграмма базы данных веб-приложения «ДебютМастер»

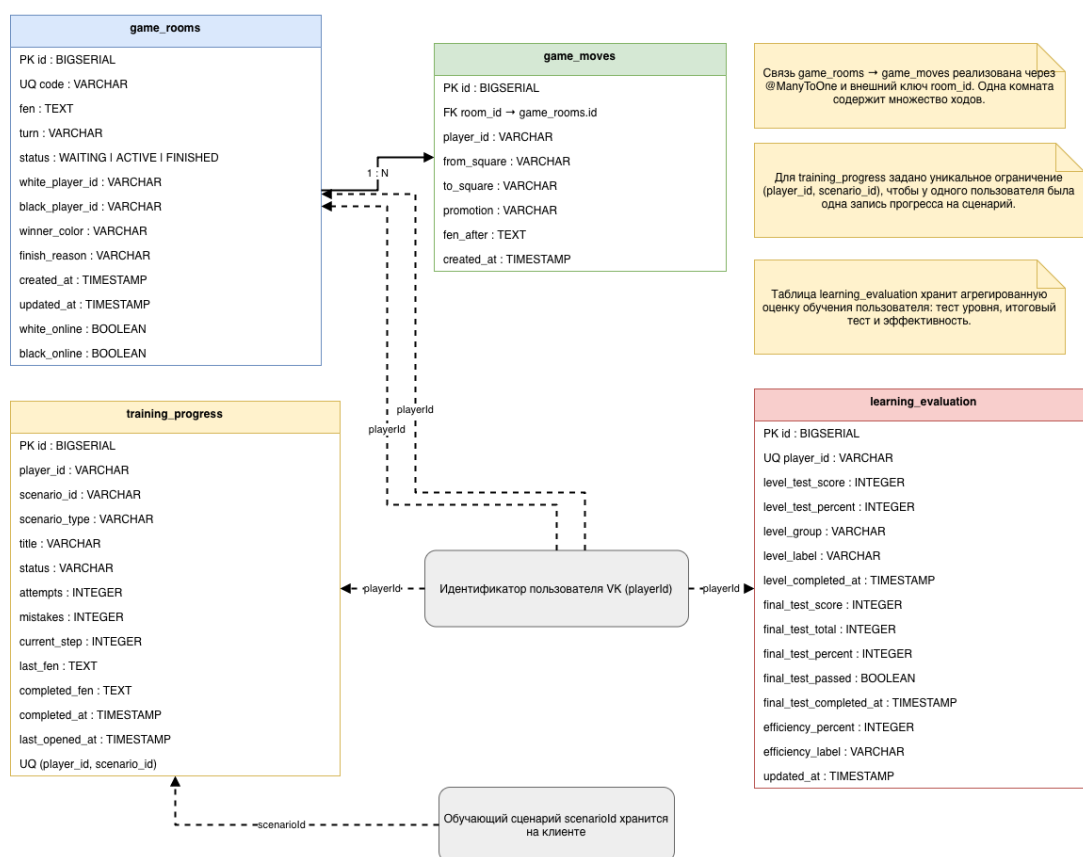


Рисунок 14 – ER-диаграмма базы данных веб-приложения «ДебютМастер»

Таблица `game_rooms` предназначена для хранения информации об игровых комнатах и текущем состоянии онлайн-партии. В таблице сохраняется уникальный код комнаты, текущая шахматная позиция в формате FEN, сторона, за которой закреплён текущий ход, статус партии, идентификаторы игроков, информация о победителе, причина завершения

партии, а также признаки нахождения игроков в сети. Статус партии может принимать значения WAITING, ACTIVE и FINISHED, что соответствует ожиданию второго игрока, активной партии и завершённой партии.

Таблица `game_moves` используется для хранения ходов, выполненных в рамках онлайн-партии. Каждый ход связан с конкретной игровой комнатой через внешний ключ `room_id`, который ссылается на поле `id` таблицы `game_rooms`. Таким образом, между таблицами `game_rooms` и `game_moves` реализована связь «один ко многим»: одна игровая комната может содержать множество ходов. В таблице `game_moves` сохраняются идентификатор игрока, начальная и конечная клетки хода, информация о превращении фигуры, позиция после выполнения хода в формате FEN и время создания записи.

Таблица `training_progress` предназначена для хранения прогресса прохождения обучающих сценариев. В ней сохраняются идентификатор пользователя, идентификатор сценария, тип сценария, название сценария, статус прохождения, количество попыток, количество ошибок, текущий шаг сценария, последняя сохранённая позиция и итоговая позиция завершённого сценария. Для таблицы задано уникальное ограничение на пару полей `player_id` и `scenario_id`. Это необходимо для того, чтобы у одного пользователя была только одна запись прогресса для каждого обучающего сценария. При повторном прохождении или продолжении сценария существующая запись обновляется.

Таблица `learning_evaluation` хранит агрегированную оценку обучения пользователя. В ней сохраняются результаты теста начального уровня, результаты итогового теста, группа и текстовое обозначение уровня пользователя, процент выполнения итогового теста, признак успешного прохождения, рассчитанный показатель эффективности обучения и его текстовая интерпретация. Для поля `player_id` задано уникальное ограничение, поскольку для одного пользователя хранится одна актуальная запись с результатами оценки обучения.

Идентификатор пользователя `playerId` поступает из платформы VK и используется в нескольких таблицах базы данных. В таблице `game_rooms` он применяется для хранения идентификаторов белого и чёрного игроков, а в таблицах `training_progress` и `learning_evaluation` — для привязки прогресса и результатов обучения к конкретному пользователю. Отдельная таблица пользователей в текущей версии базы данных не создаётся, так как приложение использует внешний идентификатор пользователя VK.

Идентификатор `scenarioId` используется для связи записи прогресса с обучающим сценарием. При этом сами обучающие сценарии хранятся на клиентской стороне приложения, поэтому отдельная таблица сценариев в базе данных не используется. В базе данных сохраняются только результаты взаимодействия пользователя со сценарием: статус, количество попыток, ошибки, текущий шаг и сохранённые позиции.

Для работы с базой данных в серверной части используется Spring Data JPA [16]. Сущности `GameRoom`, `GameMove`, `TrainingProgress` и `LearningEvaluation` соответствуют таблицам базы данных, а доступ к ним выполняется через репозитории `GameRoomRepository`, `GameMoveRepository`, `TrainingProgressRepository` и `LearningEvaluationRepository`. Такой подход позволяет отделить бизнес-логику приложения от операций непосредственного доступа к данным.

Таким образом, спроектированная структура базы данных обеспечивает хранение информации, необходимой для работы игровых и обучающих функций приложения. Таблицы `game_rooms` и `game_moves` отвечают за онлайн-партии и историю ходов, `training_progress` хранит состояние прохождения обучающих сценариев, а `learning_evaluation` используется для сохранения результатов тестирования и оценки эффективности обучения пользователя.

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА

После этапа проектирования была выполнена реализация веб-приложения «ДебютМастер», предназначенного для изучения шахматных дебютов, решения тактических задач, прохождения тестирования и игры в шахматы в нескольких режимах. Приложение включает клиентскую часть, серверную часть, модуль обучающих сценариев, модуль игрового взаимодействия, интеграцию с шахматным движком, онлайн-режим и механизм хранения пользовательского прогресса.

В рамках реализации были использованы технологии React, TypeScript, VKUI, Java, Spring Boot, WebSocket, PostgreSQL и шахматные библиотеки для обработки игровой логики. Клиентская часть отвечает за отображение интерфейса и взаимодействие пользователя с системой, а серверная часть обеспечивает хранение данных, работу онлайн-комнат, сохранение прогресса и результатов тестирования.

Далее рассмотрены основные реализованные компоненты программного продукта.

3.1 Реализация frontend-части

Frontend-часть веб-приложения «ДебютМастер» реализована с использованием React и TypeScript. Для построения пользовательского интерфейса применялась библиотека VKUI, что позволило адаптировать внешний вид приложения под стиль платформы VK Games и обеспечить единообразное отображение элементов управления.

Клиентская часть отвечает за отображение основных экранов приложения, обработку действий пользователя, навигацию между разделами, работу шахматной доски, запуск игровых режимов, прохождение обучающих сценариев, отображение тестов и визуализацию прогресса. Взаимодействие пользователя с приложением построено вокруг трёх основных разделов: игрового режима, обучающих сценариев и панели прогресса.

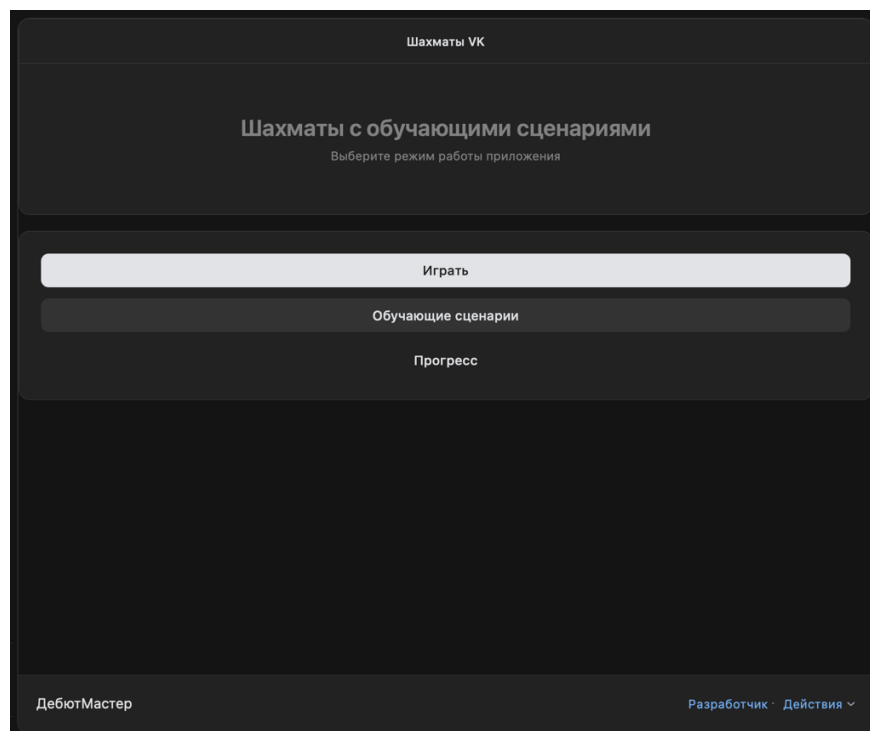


Рисунок 15 – Главный экран веб-приложения «ДебютМастер»

На рисунке 15 представлен главный экран приложения. Он содержит название приложения и основные кнопки перехода к функциональным разделам: «Играть», «Обучающие сценарии» и «Прогресс». Такая структура позволяет пользователю сразу выбрать нужный сценарий работы: перейти к партии, начать обучение или посмотреть результаты прохождения.

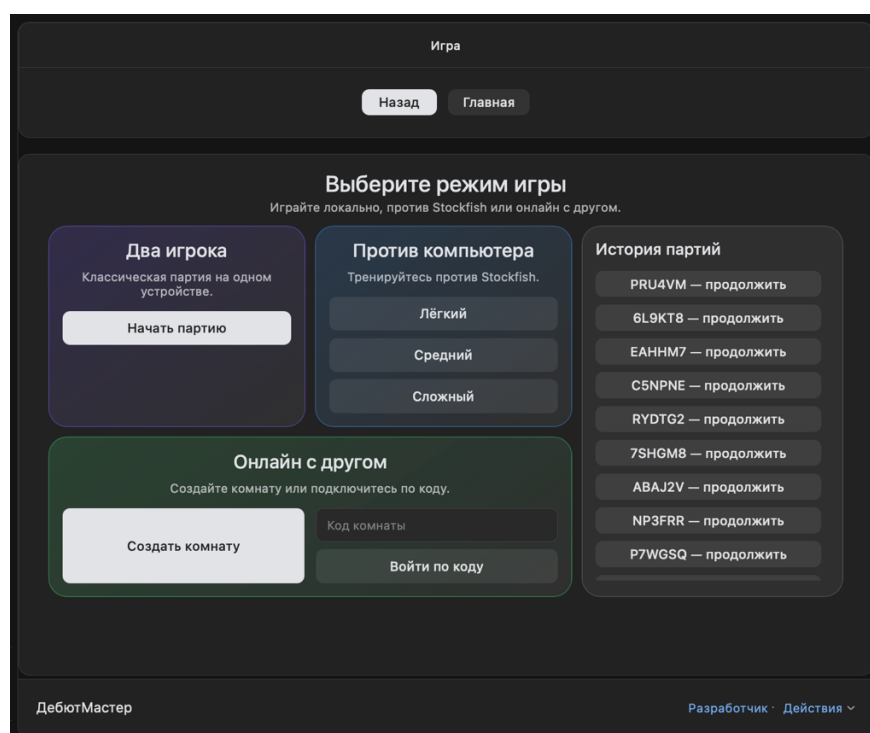


Рисунок 16 – Экран выбора режима игры

Раздел игры предоставляет пользователю несколько вариантов взаимодействия с шахматной доской. Реализованы режим локальной партии для двух игроков на одном устройстве, режим игры против компьютерного соперника, а также онлайн-режим с созданием игровой комнаты и подключением по коду. Дополнительно на экране отображается история ранее созданных партий, что позволяет пользователю продолжить незавершённую игру. Экран выбора режима игры представлен на рисунке 16.

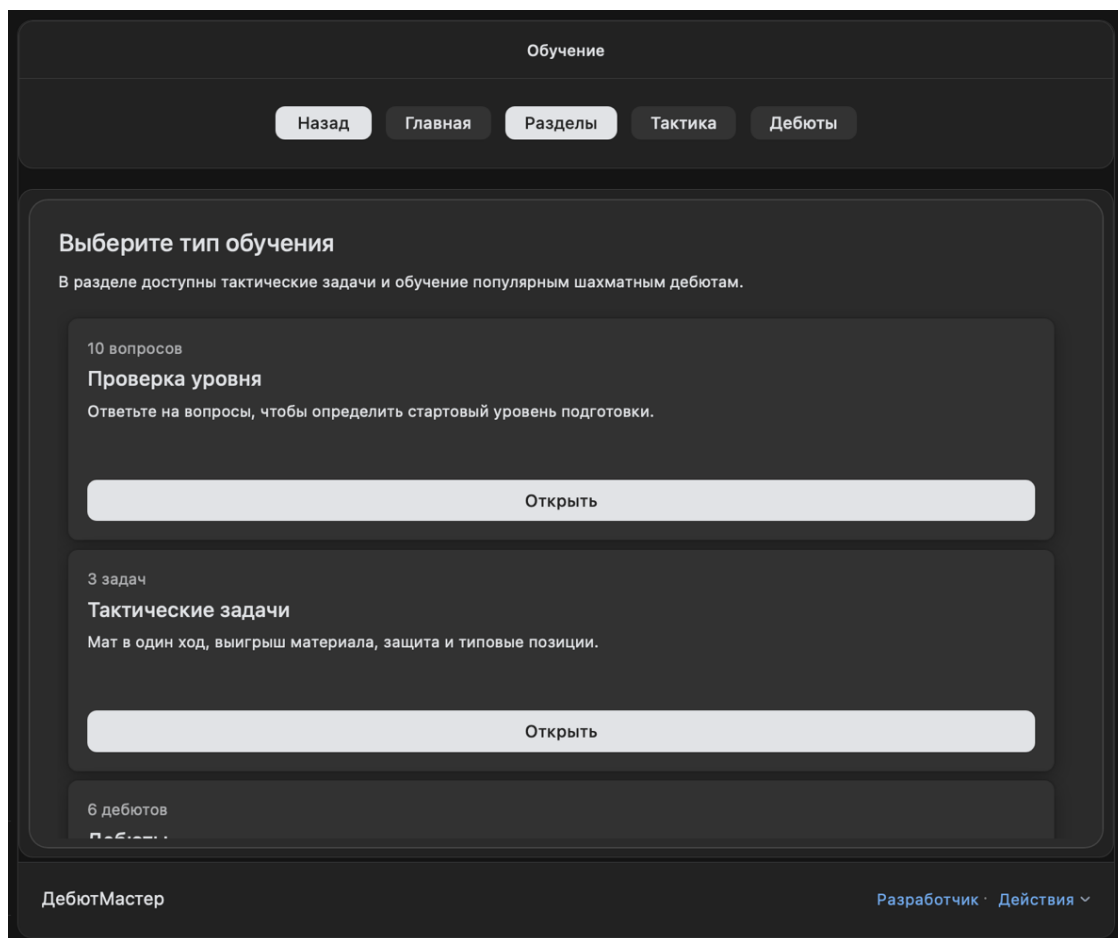


Рисунок 17 – Экран выбора типа обучения

На рисунке 17 показан экран выбора типа обучения. В данном разделе пользователь может пройти тест начального уровня, открыть список тактических задач, перейти к изучению дебютов или запустить итоговый тест. Такое разделение позволяет организовать обучение поэтапно: сначала определить уровень пользователя, затем предложить ему практические сценарии и в конце проверить усвоение материала.

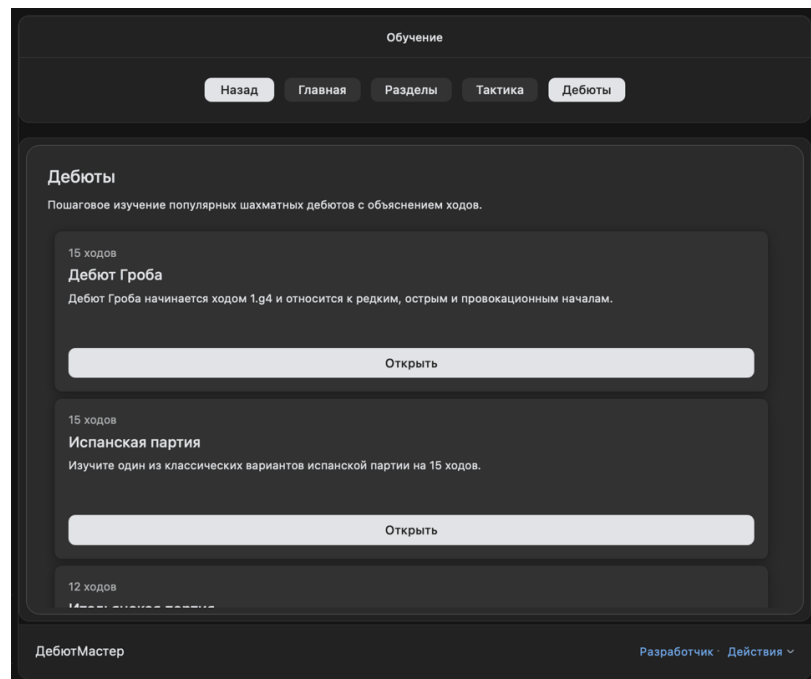


Рисунок 18 – Экран списка дебютных сценариев

Раздел дебютов содержит список доступных обучающих сценариев. Для каждого дебюта отображается название, краткое описание и количество ходов. Экран списка дебютных сценариев представлен на рисунке 18. Пользователь может выбрать интересующий дебют и перейти к его пошаговому изучению. В текущей версии приложения сценарии хранятся на клиентской стороне и включают последовательность ходов, описание дебюта и пояснения к отдельным этапам разыгрывания.

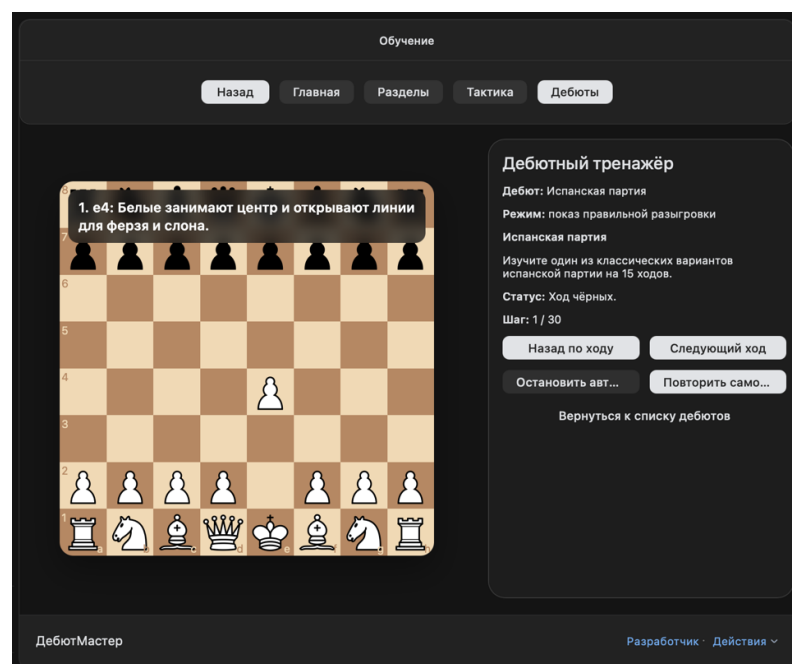


Рисунок 19 – Интерфейс прохождения дебютного сценария

На рисунке 19 представлен интерфейс прохождения дебютного сценария. Экран разделён на две основные области: шахматную доску и информационную панель. На доске отображается текущая позиция, а в правой части интерфейса выводятся название дебюта, режим прохождения, описание текущего состояния, номер шага и элементы управления.

Для дебютных сценариев реализованы два режима: демонстрация правильного разыгрывания и самостоятельное повторение. В режиме демонстрации пользователь может переходить к следующему или предыдущему ходу, а также запускать автоматический показ дебюта. При автопоказе на доске отображается пояснение к текущему ходу, что помогает пользователю понимать не только последовательность ходов, но и их смысл. После изучения демонстрации пользователь может перейти к самостоятельному повторению дебюта.

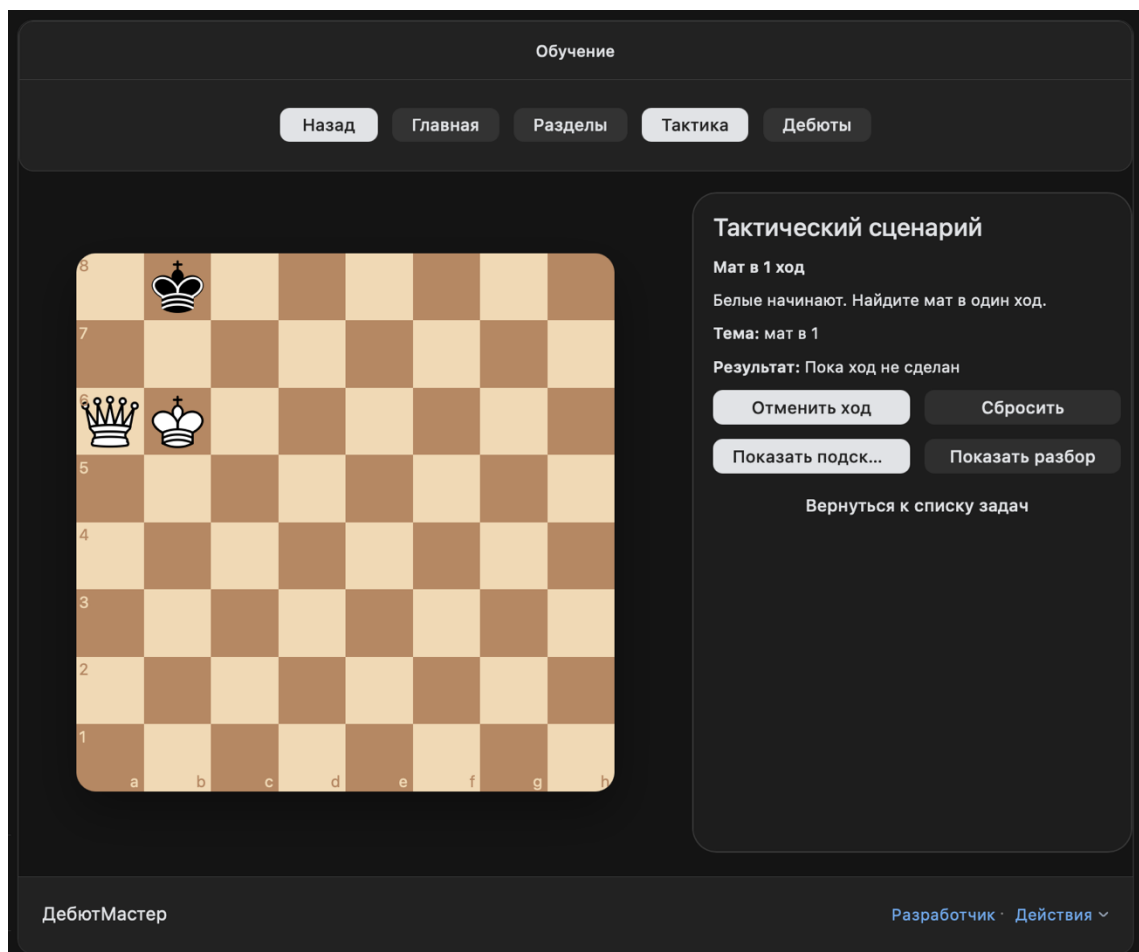


Рисунок 20 – Интерфейс решения тактической задачи

Помимо дебютных сценариев в приложении реализован раздел тактических задач. Интерфейс решения тактической задачи представлен на рисунке 20. Пользователь видит шахматную позицию и должен самостоятельно выполнить ход на доске. В правой информационной панели отображаются название задачи, описание, тема, текущий результат, а также кнопки управления.

Для тактических задач реализованы подсказки и разбор решения. Если пользователь испытывает затруднение, он может открыть подсказку. После выполнения хода система проверяет корректность решения и отображает результат. Такой подход позволяет использовать тактические задачи как дополнительный инструмент закрепления шахматных навыков.

Обучение

Назад Главная Разделы Тактика Дебюты

Проверка уровня

Ответьте на вопросы, чтобы определить стартовый уровень подготовки.

Я знаю правила перемещения всех шахматных фигур.

Да Нет

Я понимаю, что такое шах и мат.

Да Нет

Я умею читать шахматную нотацию.

Да Нет

Я уже играл шахматные партии онлайн.

Да Нет

Я играл против компьютерного соперника.

Да Нет

ДебютМастер Разработчик · Действия

Рисунок 21 – Интерфейс теста начального уровня

Для персонализации обучения в приложении реализован тест начального уровня. Как показано на рисунке 21, тест состоит из набора вопросов с вариантами ответа «Да» и «Нет». Вопросы позволяют определить, знаком ли пользователь с правилами перемещения фигур, шахматной нотацией, онлайн-

партиями, компьютерным соперником и базовыми принципами дебютной подготовки.

После завершения теста система подсчитывает количество положительных ответов и определяет уровень подготовки пользователя. В зависимости от результата пользователь относится к одной из групп: новичок, средний уровень или уверенный пользователь. Данный результат в дальнейшем используется при расчёте эффективности обучения.

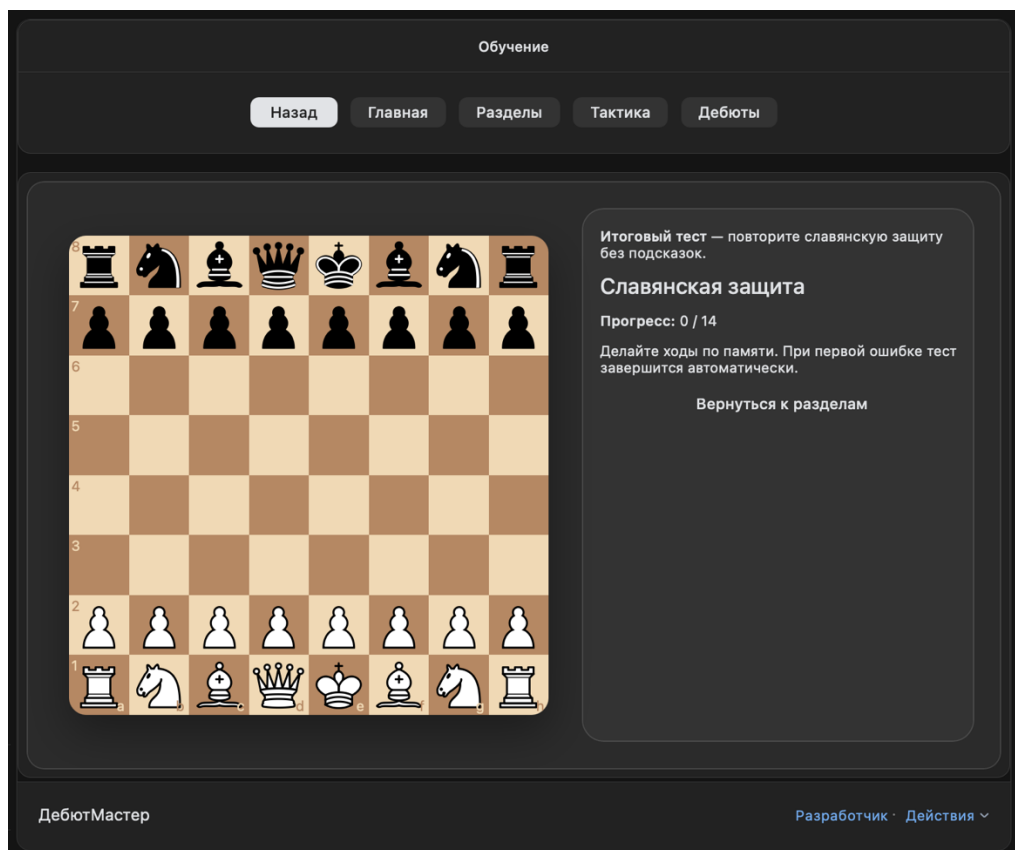


Рисунок 22 – Интерфейс итогового теста

Итоговый тест используется для проверки усвоения дебютного материала. В текущей версии приложения в качестве итогового сценария используется повторение славянской защиты без подсказок. Интерфейс итогового теста представлен на рисунке 22.

Пользователь должен выполнять ходы по памяти. В отличие от режима обучения, итоговый тест не содержит подсказок и автоматического показа. При первой ошибке прохождение завершается, после чего система сохраняет результат теста. На основе результата итогового теста и ранее определённого уровня пользователя рассчитывается показатель эффективности обучения.

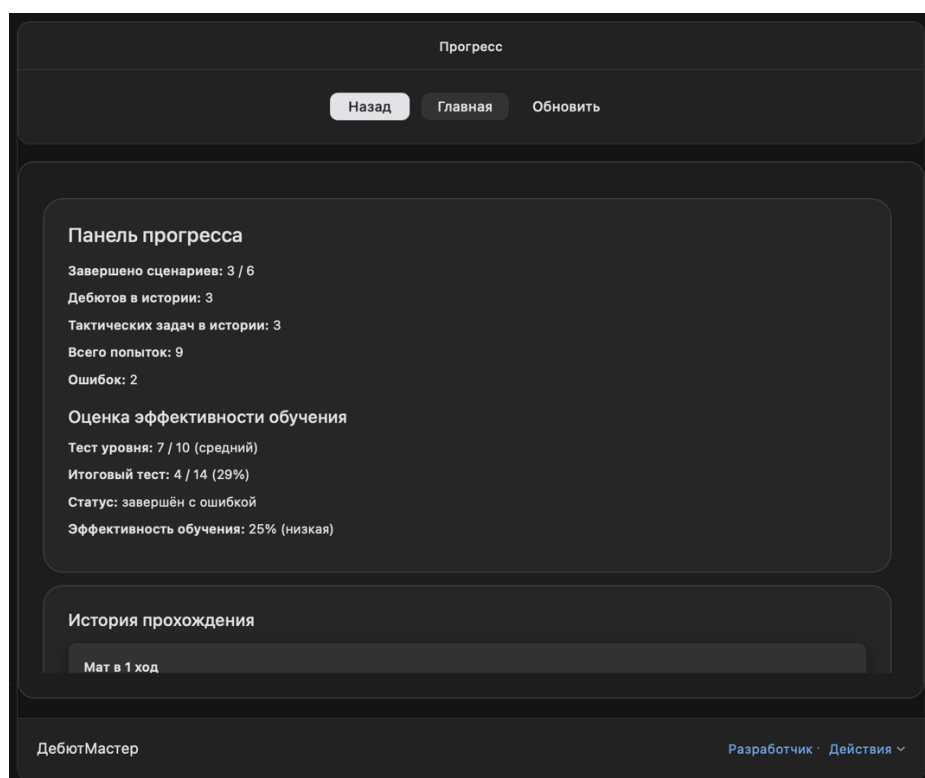


Рисунок 23 – Панель прогресса пользователя

Для отображения результатов обучения реализована панель прогресса пользователя, представленная на рисунке 23. В данном разделе отображается количество завершённых сценариев, число дебютов и тактических задач в истории, общее количество попыток и ошибок.

Отдельный блок панели прогресса содержит оценку эффективности обучения. В нём отображаются результат теста уровня, результат итогового теста, статус его прохождения и итоговый процент эффективности. Также в разделе сохраняется история прохождения сценариев. Пользователь может открыть сохранённую позицию, продолжить незавершённый сценарий или удалить запись из истории.

Таким образом, frontend-часть приложения обеспечивает полный пользовательский сценарий работы с системой: запуск приложения, выбор режима игры, прохождение обучающих сценариев, выполнение ходов на шахматной доске, получение подсказок, прохождение тестирования и просмотр прогресса. Использование React, TypeScript и VKUI позволило реализовать интерактивный интерфейс, адаптированный под размещение на платформе VK Games.

3.2 Реализация backend-части

Backend-часть веб-приложения «ДебютМастер» реализована на языке Java с использованием фреймворка Spring Boot [14, 15]. Серверная часть предназначена для обработки запросов клиентского приложения, управления онлайн-комнатами, сохранения состояния партий, хранения прогресса обучающих сценариев, сохранения результатов тестирования и передачи обновлений между пользователями в режиме реального времени.

В приложении используется слоистая структура backend-части. Основные компоненты разделены на контроллеры, сервисы, репозитории, доменные сущности, DTO-объекты и конфигурационные классы. Такое разделение позволяет отделить обработку входящих запросов от бизнес-логики и операций доступа к данным.

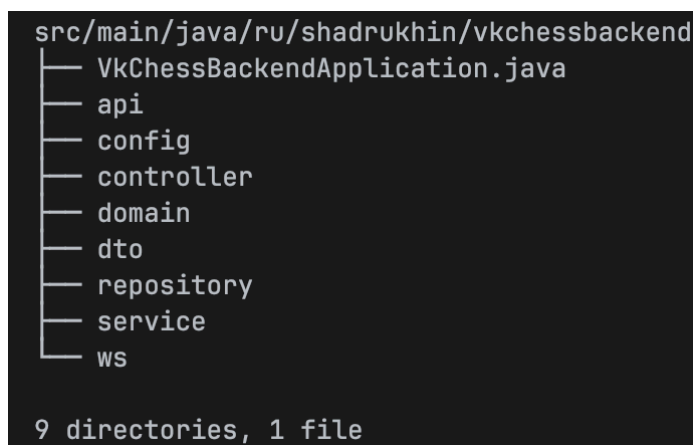


Рисунок 24 – Структура backend-части веб-приложения «ДебютМастер»

Как показано на рисунке 24, backend-часть приложения включает несколько основных пакетов. Пакет controller содержит классы, отвечающие за обработку REST-запросов и WebSocket-сообщений. Пакет service содержит бизнес-логику приложения. Пакет repository используется для доступа к данным через Spring Data JPA. Пакет domain содержит JPA-сущности, соответствующие таблицам базы данных. Пакет dto используется для описания объектов передачи данных между клиентской и серверной частями приложения. Пакет config содержит конфигурационные классы, необходимые для настройки CORS и WebSocket-взаимодействия.

Входной точкой серверного приложения является класс `VkChessBackendApplication`. Также в проекте присутствует пакет `api`, содержащий `HealthController`, который используется для проверки доступности серверной части.

Для обработки запросов, связанных с игровыми комнатами, реализован контроллер `GameRoomController`. Он предоставляет REST API для создания комнаты, получения информации о комнате, подключения пользователя, выхода из комнаты, завершения партии и получения списка партий пользователя.

```
1 package ru.shadrukhin.vkchessbackend.controller;
2
3 > import ...
4
14
15 @RestController & AlexandrShadruxhin
16 @RequestMapping("/api/rooms")
17 @RequiredArgsConstructor
18 @CrossOrigin(origins = "*")
19 public class GameRoomController {
20
21     private final GameRoomService gameRoomService;
22     private final SimpMessagingTemplate messagingTemplate;
23
24     @PostMapping & AlexandrShadruxhin
25     public RoomResponse createRoom(@RequestBody CreateRoomRequest request) {...}
26
27
28
29     @GetMapping("/{code}") & AlexandrShadruxhin
30     public RoomResponse getRoom(@PathVariable String code) { return gameRoomService.getRoom(code); }
31
32
33
34     @PostMapping("/{code}/join") & AlexandrShadruxhin
35     public RoomResponse joinRoom(
36         @PathVariable String code,
37         @RequestBody JoinRoomRequest request
38     ) {...}
39
40
41
42
43
44     @PostMapping("/{code}/leave") & AlexandrShadruxhin
45     public RoomResponse leaveRoom(
46         @PathVariable String code,
47         @RequestBody LeaveRoomRequest request
48     ) {...}
49
50
51
52
53
54     @PostMapping("/{code}/finish") & AlexandrShadruxhin
55     public RoomResponse finishRoom(
56         @PathVariable String code,
57         @RequestBody FinishRoomRequest request
58     ) {...}
59
60
61
62
63
64
65     @GetMapping("/player/{playerId}") & AlexandrShadruxhin
66     public List<RoomResponse> getPlayerRooms(@PathVariable String playerId) {...}
67
68 }
```

Рисунок 25 – Фрагмент реализации REST API для работы с игровыми комнатами

На рисунке 25 представлен фрагмент реализации контроллера игровых комнат. Контроллер использует базовый путь `/api/rooms`. Метод `createRoom` обрабатывает POST-запрос на создание новой онлайн-комнаты. Метод `getRoom` возвращает текущее состояние комнаты по её коду. Метод `joinRoom`

используется для подключения пользователя к существующей комнате, а метод `leaveRoom` фиксирует выход пользователя из онлайн-партии.

Метод `finishRoom` применяется для завершения партии и сохранения результата. Метод `getPlayerRooms` возвращает список комнат, в которых участвовал пользователь. Это используется на клиентской стороне для отображения истории партий и возможности продолжения ранее начатых игр.

После подключения, выхода или завершения партии контроллер выполняет рассылку обновлённого состояния комнаты через `SimpMessagingTemplate`. Благодаря этому подключённые пользователи получают актуальное состояние онлайн-партии без необходимости обновлять страницу вручную.

Основная бизнес-логика работы с игровыми комнатами вынесена в сервис `GameRoomService`. Данный сервис отвечает за создание новой комнаты, генерацию уникального кода, подключение игроков, сохранение ходов, обновление шахматной позиции, выход пользователя из комнаты и завершение партии.

```
17 @Service & AlexandrShadrukhin
18 @RequiredArgsConstructor
19 public class GameRoomService {
20
21     private static final String INITIAL_FEN = 1 usage
22     | "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1";
23
24     private final GameRoomRepository roomRepository;
25     private final GameMoveRepository moveRepository;
26
27     private final SecureRandom random = new SecureRandom(); 1 usage
28
29     @Transactional 1 usage & AlexandrShadrukhin
30     public RoomResponse createRoom(String playerId) {
31         GameRoom room = new GameRoom();
32
33         room.setCode(generateRoomCode());
34         room.setFen(INITIAL_FEN);
35         room.setTurn("w");
36         room.setStatus(GameStatus.WAITING);
37         room.setWhitePlayerId(playerId);
38         room.setWhiteOnline(true);
39         room.setBlackOnline(false);
40         room.setCreatedAt(Instant.now());
41         room.setUpdatedAt(Instant.now());
42
43         return toResponse(roomRepository.save(room));
44     }
45
46     @Transactional 1 usage & AlexandrShadrukhin
47     > public RoomResponse joinRoom(String code, String playerId) {...}
48
49     @Transactional(readOnly = true) 1 usage & AlexandrShadrukhin
50     > public RoomResponse getRoom(String code) { return toResponse(getRoomByCode(code)); }
51
52     @Transactional 1 usage & AlexandrShadrukhin
53     @ > public RoomResponse applyMove(String code, MoveRequest request) {...}
54
55     @ > private GameRoom getRoomByCode(String code) {...}
56
57     @ > private RoomResponse toResponse(GameRoom room) { 7 usages & AlexandrShadrukhin
```

Рисунок 26 – Фрагмент реализации сервиса игровых комнат

На рисунке 26 показан фрагмент реализации сервиса игровых комнат. При создании комнаты формируется начальная шахматная позиция в формате FEN, устанавливается статус WAITING, а пользователь, создавший комнату, назначается игроком за белые фигуры. Также фиксируются дата создания, дата обновления и онлайн-статус игрока.

Для генерации кода комнаты используется отдельный метод `generateRoomCode`. Код формируется из набора символов, после чего выполняется проверка его уникальности через `GameRoomRepository`. Это позволяет исключить совпадение кодов разных онлайн-комнат.

При подключении второго пользователя метод `joinRoom` устанавливает его идентификатор как игрока за чёрные фигуры и переводит комнату в статус ACTIVE. Если пользователь повторно подключается к существующей комнате, обновляется его онлайн-статус. Это позволяет отображать в интерфейсе состояние соперника: находится он в сети или нет.

Метод `applyMove` используется при выполнении хода в онлайн-партии. Он сохраняет ход в таблицу `game_moves`, обновляет текущую позицию комнаты в формате FEN, определяет сторону следующего хода и фиксирует время обновления. Таким образом, сервер хранит не только текущее состояние партии, но и историю ходов.

Кроме игрового режима, backend-часть реализует хранение данных, связанных с обучением пользователя. Для этого используются сервисы `TrainingProgressService` и `LearningEvaluationService`. Первый сервис отвечает за сохранение прогресса прохождения обучающих сценариев, а второй — за хранение результатов теста уровня, итогового теста и рассчитанной эффективности обучения.

```

13 @Service & AlexandrShadrukhin
14 @RequiredArgsConstructor
15 public class LearningEvaluationService {
16
17     private final LearningEvaluationRepository repository;
18
19     @Transactional(readOnly = true) 1 usage & AlexandrShadrukhin
20     public LearningEvaluationResponse getEvaluation(String playerId) {
21         return repository.findByPlayerId(playerId).Optional<LearningEvaluation>
22             .map(this::toResponse).Optional<LearningEvaluationResponse>
23             .orElseGet(() -> emptyResponse(playerId));
24     }
25
26     @Transactional 1 usage & AlexandrShadrukhin
27     @ public LearningEvaluationResponse saveEvaluation(LearningEvaluationRequest request) {
28         LearningEvaluation evaluation = repository
29             .findByPlayerId(request.playerId())
30             .orElseGet(LearningEvaluation::new);
31
32         Instant now = Instant.now();
33
34         evaluation.setPlayerId(request.playerId());
35
36         > if (request.levelTestScore() != null) {...}
39         > if (request.levelTestPercent() != null) {...}
42         > if (request.levelGroup() != null) {...}
45         > if (request.levelLabel() != null) {...}
48         > if (hasLevelData(request)) {...}
51
52         > if (request.finalTestScore() != null) {...}
55         > if (request.finalTestTotal() != null) {...}
58         > if (request.finalTestPercent() != null) {...}
61         > if (request.finalTestPassed() != null) {...}
64         > if (hasFinalData(request)) {...}
67
68         > if (request.efficiencyPercent() != null) {...}
71         > if (request.efficiencyLabel() != null) {...}
74
75         evaluation.setUpdatedAt(now);
76
77         return toResponse(repository.save(evaluation));

```

Рисунок 27 – Фрагмент реализации сервиса сохранения результатов обучения

На рисунке 27 представлен фрагмент реализации сервиса `LearningEvaluationService`. Метод `getEvaluation` получает сохранённую оценку обучения пользователя по его идентификатору. Если запись отсутствует, возвращается пустой объект ответа. Метод `saveEvaluation` сохраняет или обновляет данные об уровне подготовки пользователя, результате итогового теста и эффективности обучения.

При сохранении результата система проверяет наличие данных теста начального уровня, итогового теста и показателя эффективности. Если соответствующие данные присутствуют в запросе, они записываются в сущность `LearningEvaluation`. Также фиксируется время обновления записи.

Такой подход позволяет хранить одну актуальную агрегированную оценку обучения для каждого пользователя.

Для хранения прогресса отдельных обучающих сценариев используется сервис `TrainingProgressService`. Он сохраняет идентификатор пользователя, идентификатор сценария, тип сценария, название, статус прохождения, количество попыток, количество ошибок, текущий шаг, последнюю позицию и итоговую позицию. Если пользователь повторно проходит сценарий или продолжает незавершённое обучение, существующая запись обновляется.

Для доступа к базе данных используются репозитории `Spring Data JPA`: `GameRoomRepository`, `GameMoveRepository`, `TrainingProgressRepository` и `LearningEvaluationRepository` [16]. Они обеспечивают выполнение операций поиска, сохранения, обновления и удаления записей в базе данных `PostgreSQL`.

Развертывание backend-части выполняется с использованием `Dockerfile`, описывающего запуск серверного приложения в контейнере [20].

Передача данных между клиентской и серверной частями выполняется с использованием DTO-объектов. Для создания комнаты используется `CreateRoomRequest`, для подключения к комнате — `JoinRoomRequest`, для передачи хода — `MoveRequest`, для сохранения прогресса — `TrainingProgressRequest`, а для сохранения результатов обучения — `LearningEvaluationRequest`. Ответы сервера формируются с использованием `RoomResponse`, `TrainingProgressResponse` и `LearningEvaluationResponse`.

Таким образом, backend-часть веб-приложения «ДебютМастер» обеспечивает основные серверные функции системы: управление онлайн-комнатами, хранение состояния партий, хранение истории ходов, хранение прогресса обучающих сценариев, хранение результатов тестирования и предоставление данных клиентской части через `REST API` и `WebSocket`.

3.3 Интеграция шахматного движка

Для реализации режима игры против компьютерного соперника в веб-приложение «ДебютМастер» был интегрирован шахматный движок `Stockfish`

[12, 13]. Использование шахматного движка позволяет автоматически анализировать текущую позицию на доске и формировать ответный ход за компьютерного соперника.

Интеграция движка необходима для того, чтобы пользователь мог тренироваться не только в обучающих сценариях, но и в формате полноценной шахматной партии. Такой режим позволяет закреплять навыки принятия решений, полученные при изучении дебютов и решении тактических задач.

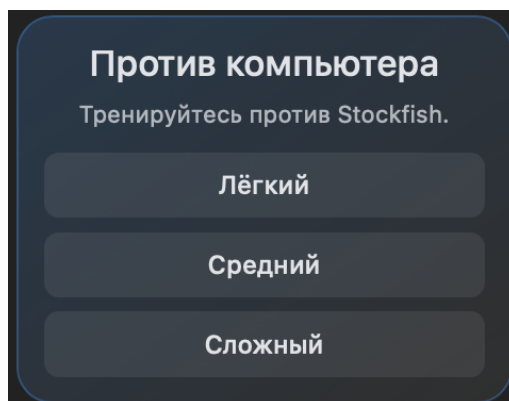


Рисунок 28 – Выбор сложности игры против компьютерного соперника

Как показано на рисунке 28, пользователь может выбрать один из трёх уровней сложности компьютерного соперника: лёгкий, средний или сложный. Каждый уровень соответствует разной глубине анализа шахматной позиции. Чем выше сложность, тем больше глубина поиска хода и тем сильнее играет компьютерный соперник.

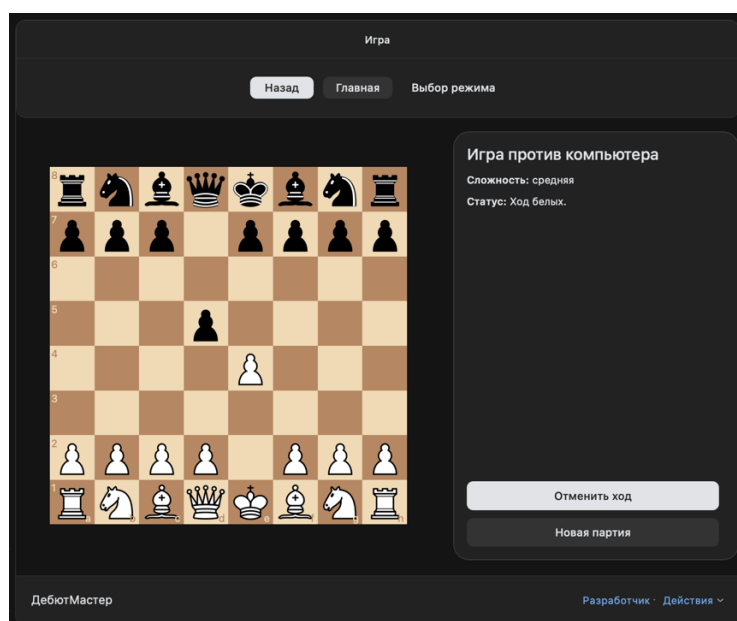


Рисунок 29 – Интерфейс игры против компьютерного соперника

На рисунке 29 представлен интерфейс партии против компьютерного соперника. В левой части экрана расположена шахматная доска, а в правой части — информационная панель с названием режима, выбранной сложностью и текущим статусом партии.

Пользователь выполняет ход на шахматной доске за белые фигуры. После этого текущее состояние партии обновляется, позиция передаётся шахматному движку, и приложение ожидает ответный ход компьютера. Полученный ход применяется к текущей партии, после чего доска и статус игры обновляются в пользовательском интерфейсе.

```
1  export type BotDifficulty = 'easy' | 'medium' | 'hard';
2
3  const STOCKFISH_WORKER_URL = '/stockfish/stockfish.js';
4
5  const difficultyDepth: Record<BotDifficulty, number> = {
6    easy: 2,
7    medium: 6,
8    hard: 10,
9  };
10
11 let engine: Worker | null = null;
12 let initialized: boolean = false;
13
14 > const getEngine: () => Worker = () => { ... };
21
22 > const waitForMessage: (expected: string, timeoutMs?: number) => ... = (expected: string, timeoutMs = 5000) : Promise<void> => { ... };
44
45 > const initEngine: () => Promise<void> = async () : Promise<void> => { ... };
60
61 export const getStockfishMove: (fen: string, difficulty: BotDifficulty) ... = async ( Show usages  AlexandrShadruxhin
62   fen: string,
63   difficulty: BotDifficulty,
64 ): Promise<string | null> => {
65   await initEngine();
66
67   const worker: Worker = getEngine();
68   const depth: number = difficultyDepth[difficulty];
69
70   return new Promise((resolve: (value: string | PromiseLike<string | nul... ) : void => {
71     const timeout: number = window.setTimeout(() : void => { ... }, timeout: 10000);
75
76     const handler: (event: MessageEvent<any>) => void = (event: MessageEvent) : void => { ... };
89
90     worker.addEventListener( type: 'message', handler);
91
92     worker.postMessage( message: `position fen ${fen}`);
93     worker.postMessage( message: `go depth ${depth}`);
94   });
95 }
```

Рисунок 30 — Фрагмент реализации взаимодействия с шахматным движком Stockfish

На рисунке 30 представлен фрагмент сервиса stockfishEngine.ts, отвечающего за взаимодействие с шахматным движком Stockfish. В сервисе задаётся путь к файлу движка, размещённому по адресу /stockfish/stockfish.js, а также определяется соответствие между выбранной пользователем сложностью и глубиной анализа позиции.

Для лёгкого уровня используется глубина анализа 2, для среднего — 6, для сложного — 10. Таким образом, сложность компьютерного соперника регулируется не отдельными наборами правил, а параметром глубины поиска хода.

Для запуска Stockfish используется объект Worker. Данный подход позволяет выполнять вычисления шахматного движка в отдельном потоке и не блокировать пользовательский интерфейс приложения. При первом обращении к движку создаётся экземпляр Worker, после чего выполняется его инициализация.

Основная функция `getStockfishMove` принимает текущую позицию в формате FEN и выбранный уровень сложности. Позиция передаётся движку командой `position fen`, затем запускается расчёт хода командой `go depth`. После завершения анализа Stockfish возвращает сообщение `bestmove`, из которого извлекается лучший ход. Полученный ход передаётся обратно в игровой компонент и применяется к текущей партии.

Также в реализации предусмотрен таймаут ожидания ответа от движка. Если Stockfish не возвращает ход за заданное время, функция возвращает `null`. Это позволяет избежать зависания игрового интерфейса при ошибке работы движка или невозможности найти ход.

Таким образом, интеграция шахматного движка Stockfish позволила реализовать режим игры против компьютерного соперника с несколькими уровнями сложности. Использование формата FEN и протокола взаимодействия с движком через Web Worker обеспечивает корректную передачу текущей позиции, получение ответного хода и обновление состояния партии без блокировки интерфейса пользователя.

3.4 Реализация онлайн-игры

В веб-приложении «ДебютМастер» реализован режим онлайн-игры, позволяющий двум пользователям играть шахматную партию через общую игровую комнату. Данный режим дополняет обучающую часть приложения и

предоставляет пользователю возможность применить полученные знания в игровой ситуации с другим игроком.

Онлайн-игра построена на основе механизма игровых комнат. Один пользователь создаёт комнату, после чего приложение формирует уникальный код. Второй пользователь может подключиться к партии, указав данный код в соответствующем поле. После подключения обоих игроков комната переводится в активное состояние, а пользователи получают возможность выполнять ходы по очереди.

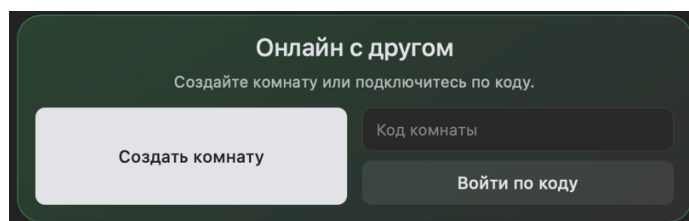


Рисунок 31 – Выбор онлайн-режима игры

На рисунке 31 представлен блок выбора онлайн-режима. Пользователь может создать новую комнату или подключиться к существующей партии по коду. Такой подход позволяет организовать простое подключение второго игрока без необходимости поиска пользователя по имени или ручной настройки соединения.

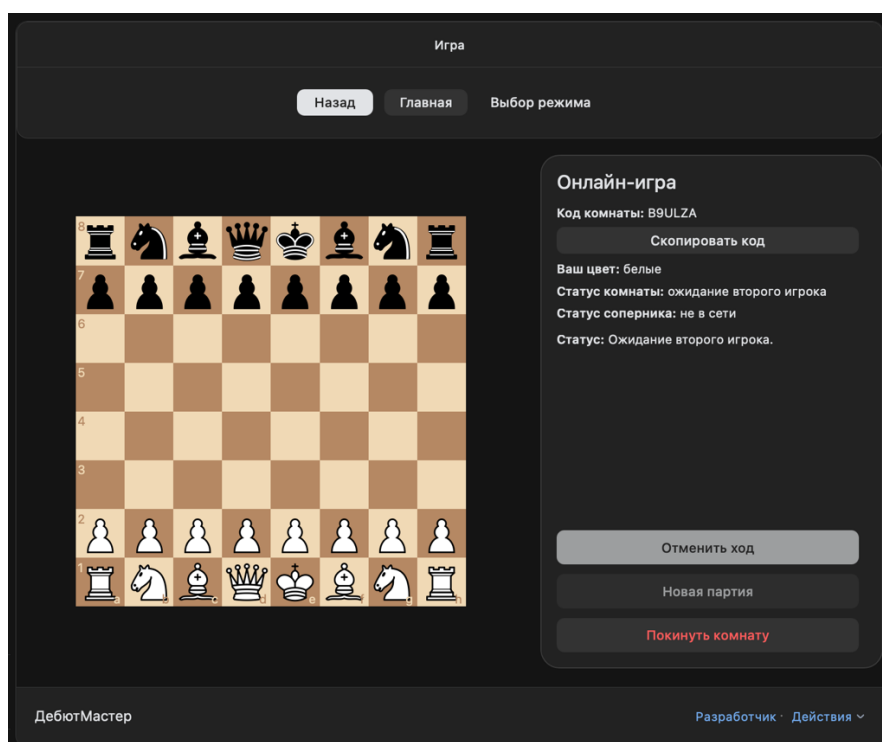


Рисунок 32 – Интерфейс созданной онлайн-комнаты

После создания комнаты пользователь переходит к экрану онлайн-партии. Как показано на рисунке 32, в правой части интерфейса отображается код комнаты, кнопка копирования кода, цвет текущего игрока, статус комнаты, статус соперника и общий статус партии. Если второй пользователь ещё не подключился, система отображает состояние ожидания второго игрока.

Пользователь, создавший комнату, получает белые фигуры. До подключения второго игрока партия находится в режиме ожидания. Кнопка «Скопировать код» предназначена для передачи кода комнаты другому пользователю, который может ввести его на экране выбора режима игры.

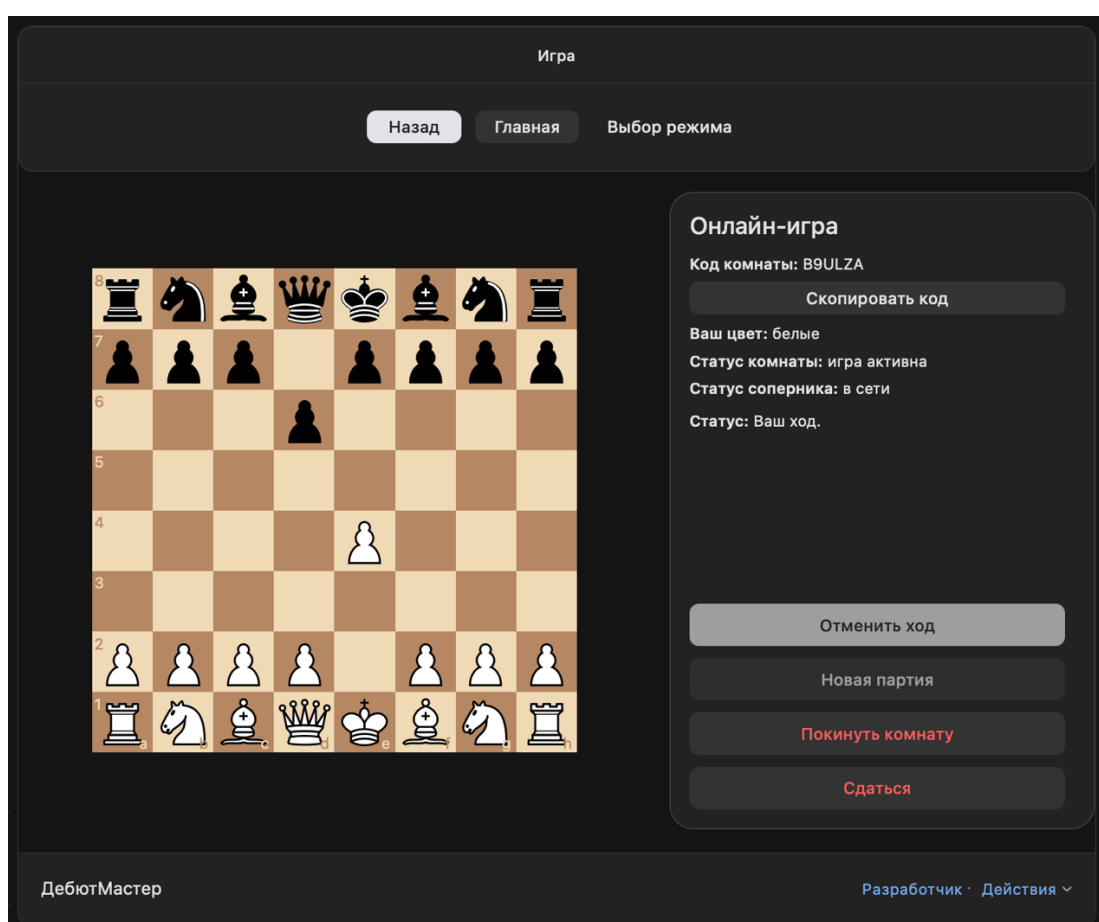


Рисунок 33 – Интерфейс активной онлайн-партии

После подключения второго игрока комната переходит в активное состояние. На рисунке 33 представлен интерфейс активной онлайн-партии. В информационной панели отображается код комнаты, цвет игрока, состояние комнаты и текущий статус хода. Пользователь может выполнять ход на

шахматной доске, после чего обновлённое состояние партии передаётся второму участнику.

В процессе онлайн-игры приложение отслеживает текущую позицию партии, сторону хода, онлайн-статус игроков и состояние комнаты. Также пользователю доступны действия отмены хода, выхода из комнаты и сдачи партии.

```
1 package ru.shadrukhin.vkchessbackend.config;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.messaging.simp.config.MessageBrokerRegistry;
5 import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
6 import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
7 import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;
8
9 @Configuration
10 @EnableWebSocketMessageBroker
11 public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
12
13     @Override
14     public void configureMessageBroker(MessageBrokerRegistry registry) {
15         registry.enableSimpleBroker("/topic");
16         registry.setApplicationDestinationPrefixes("/app");
17     }
18
19     @Override
20     public void registerStompEndpoints(StompEndpointRegistry registry) {
21         registry.addEndpoint("/ws/game")
22             .setAllowedOriginPatterns("*");
23     }
24 }
```

Рисунок 34 – Конфигурация WebSocket-взаимодействия

Для передачи обновлений между игроками в режиме реального времени используется WebSocket-взаимодействие [17]. На рисунке 34 представлен фрагмент конфигурации WebSocket. Класс `WebSocketConfig` помечен аннотацией `@EnableWebSocketMessageBroker` и реализует интерфейс `WebSocketMessageBrokerConfigurer`.

В методе `configureMessageBroker` задаётся простой брокер сообщений с префиксом `/topic`. Через данный префикс клиенты получают обновления состояния комнаты. Также задаётся префикс `/app`, используемый для сообщений, отправляемых клиентом на сервер [17]. В методе `registerStompEndpoints` регистрируется endpoint `/ws/game`, через который клиентская часть подключается к WebSocket-соединению.

Использование WebSocket позволяет не выполнять постоянные HTTP-запросы для проверки состояния партии. После выполнения хода одним

игроком сервер отправляет обновлённое состояние комнаты всем клиентам, подписанным на соответствующий канал комнаты.

Обработка хода в онлайн-партии выполняется в контроллере `GameSocketController`. Клиент отправляет сообщение на адрес `/app/rooms/{code}/move`, где `code` является кодом игровой комнаты. Сервер получает ход, вызывает метод `applyMove` сервиса `GameRoomService`, сохраняет обновлённую позицию и затем отправляет новое состояние комнаты в канал `/topic/rooms/{code}`. Благодаря этому оба участника партии получают одинаковое актуальное состояние доски.

Таким образом, онлайн-режим в веб-приложении «ДебютМастер» реализует полный сценарий сетевой партии: создание комнаты, подключение второго игрока по коду, отображение статусов игроков, передачу ходов и обновление состояния доски в реальном времени. Совместное использование REST API и WebSocket позволяет разделить операции управления комнатой и передачу игровых событий, что делает реализацию более удобной для сопровождения и дальнейшего расширения.

3.5 Тестирование

В данном разделе рассматривается функциональное тестирование разработанного веб-приложения «ДебютМастер». Тестирование проводилось с целью проверки корректности работы основных пользовательских сценариев: прохождения теста начального уровня, выполнения итогового теста, сохранения прогресса пользователя, отображения истории прохождения, а также развертывания клиентской и серверной частей приложения.

В рамках проверки были протестированы следующие функциональные возможности приложения: прохождение теста для определения начального уровня подготовки пользователя, выполнение итогового теста по изученному дебютному сценарию, сохранение результатов тестирования, расчет эффективности обучения на основе результата итогового теста и уровня

пользователя, отображение статистики и истории прохождения в панели прогресса, а также разворачивание frontend- и backend-частей приложения.

Для проверки персонализации обучения в приложении был реализован тест начального уровня. Пользователь отвечает на вопросы, связанные с базовыми знаниями правил шахмат, пониманием шахматной нотации, опытом онлайн-игр и знанием принципов дебютной игры. По результатам тестирования система определяет уровень подготовки пользователя, который далее используется при расчете эффективности обучения.

Обучение

Назад Главная Разделы Тактика Дебюты

Да Нет

Я раньше изучал шахматные дебюты.

Да Нет

Я понимаю основные принципы борьбы за центр.

Да Нет

Я понимаю принцип развития фигур в дебюте.

Да Нет

Я могу повторить знакомый дебют без подсказок.

Да Нет

Завершить тест

Ответов: 10 / 10

ДебютМастер

Разработчик · Действия

Рисунок 35 – Прохождение теста начального уровня пользователя

На рисунке 35 представлен экран прохождения теста начального уровня. В рамках данного теста пользователь отвечает на десять вопросов, после чего система рассчитывает количество положительных ответов и относит пользователя к одной из групп подготовки. В рассматриваемом примере пользователь набрал 5 баллов из 10, что соответствует среднему уровню подготовки.

После прохождения обучающих сценариев пользователь может выполнить итоговый тест. Итоговый тест предназначен для проверки того, насколько пользователь запомнил дебютную последовательность. В отличие от режима демонстрации дебюта, итоговый тест выполняется без подсказок: пользователь должен самостоятельно воспроизвести ходы изученного дебюта. При первой ошибке тест завершается, а результат сохраняется в системе прогресса.



Рисунок 36 – Прохождение итогового теста по дебютному сценарию

На рисунке 36 показан результат прохождения итогового теста по дебютному сценарию «Славянская защита». Пользователь воспроизвел 8 ходов из 14, после чего допустил ошибку. Итоговый результат составил 57%. После завершения теста приложение сохраняет результат в системе прогресса пользователя.

На основе результата теста начального уровня и результата итогового тестирования система автоматически рассчитывает показатель эффективности обучения. Алгоритм расчета был описан в теоретической части работы: итоговый процент прохождения теста корректируется с учетом начального

уровня подготовки пользователя. Такой подход позволяет оценивать результат не только по количеству правильно выполненных ходов, но и с учетом того, насколько сложным данный результат является для пользователя конкретного уровня.

В рассматриваемом примере пользователь был отнесен к среднему уровню подготовки, так как набрал 5 баллов из 10 в тесте начального уровня. После прохождения итогового теста по Славянской защите результат составил 8 ходов из 14, то есть 57%. С учетом коэффициента для среднего уровня подготовки система рассчитала эффективность обучения как 48%. Данное значение относится к средней эффективности обучения.

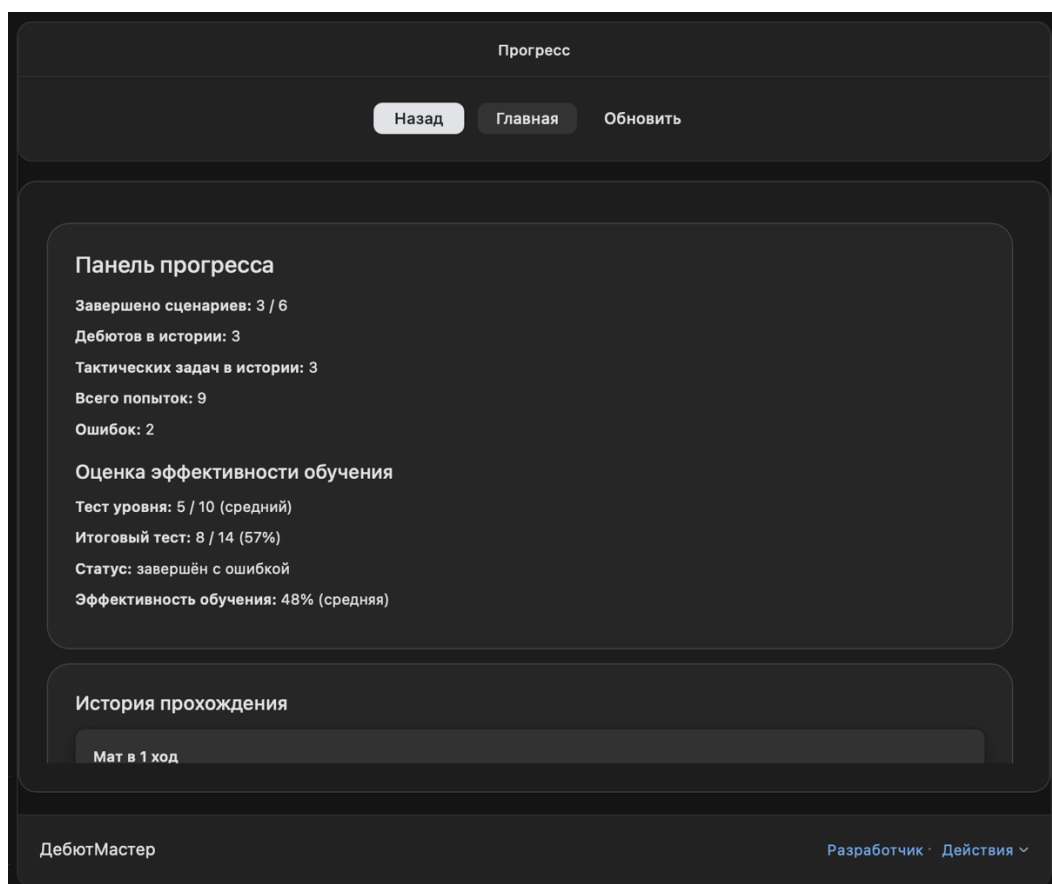


Рисунок 37 – Отображение результатов тестирования и эффективности обучения

На рисунке 37 представлена панель прогресса пользователя после прохождения тестирования. В ней отображается общее количество завершенных сценариев, число пройденных дебютов и тактических задач, количество попыток и ошибок, а также блок оценки эффективности обучения.

В приведенном примере отображаются результат теста уровня, результат итогового теста, статус прохождения и рассчитанная эффективность обучения. Это подтверждает, что данные тестирования сохраняются и используются приложением для формирования персонализированной оценки прогресса пользователя.

Дополнительно в приложении реализована история прохождения. Она позволяет пользователю просматривать ранее начатые или завершенные сценарии, открывать сохраненную позицию, продолжать незавершенное прохождение или удалять запись из истории. Наличие истории прохождения необходимо для повторной тренировки ошибочных или незавершенных сценариев.

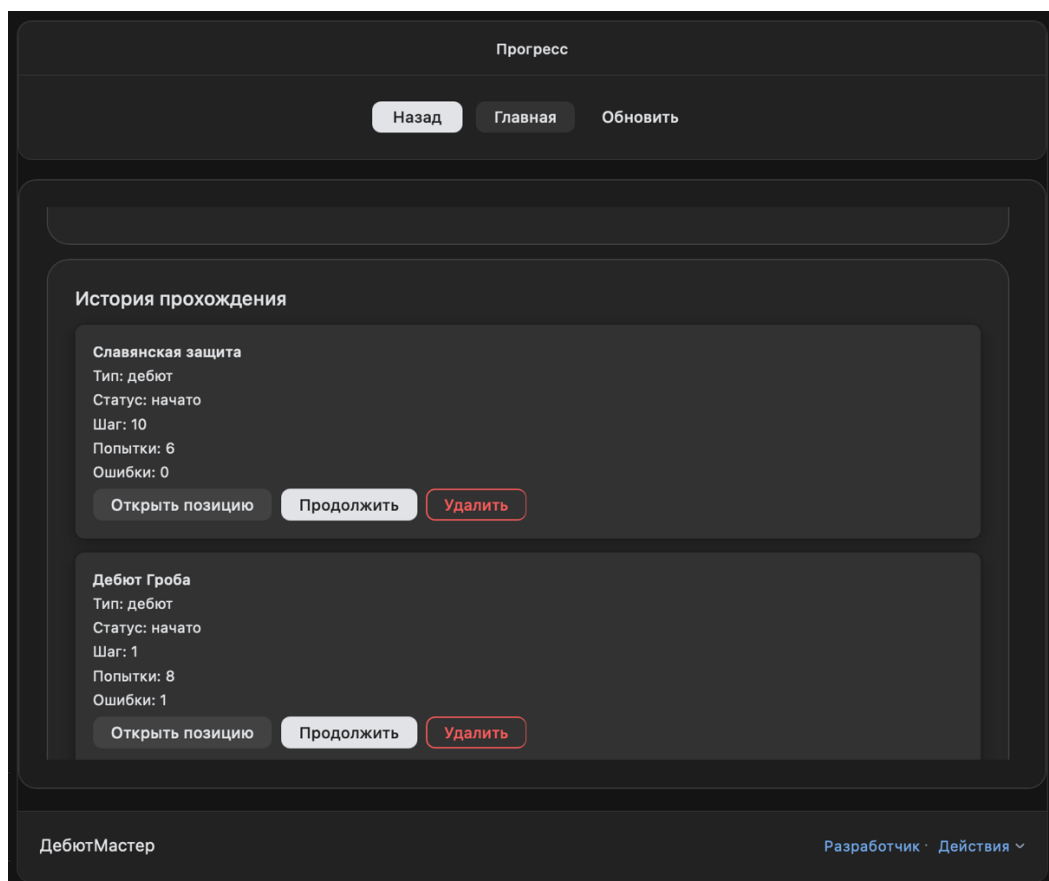


Рисунок 38 – История прохождения обучающих сценариев

На рисунке 38 показана история прохождения обучающих сценариев. Для каждой записи отображается название сценария, тип сценария, текущий статус, номер шага, количество попыток и ошибок. Пользователь может открыть сохраненную позицию, продолжить сценарий или удалить запись из

истории. Таким образом, приложение обеспечивает не только разовое прохождение заданий, но и возможность возвращаться к ранее изученным материалам.

Для проверки корректности работы приложения также была выполнена проверка развертывания. Клиентская часть веб-приложения была опубликована на платформе Vercel [21]. На странице развертывания отображается статус Ready, что подтверждает успешную публикацию frontend-части приложения.

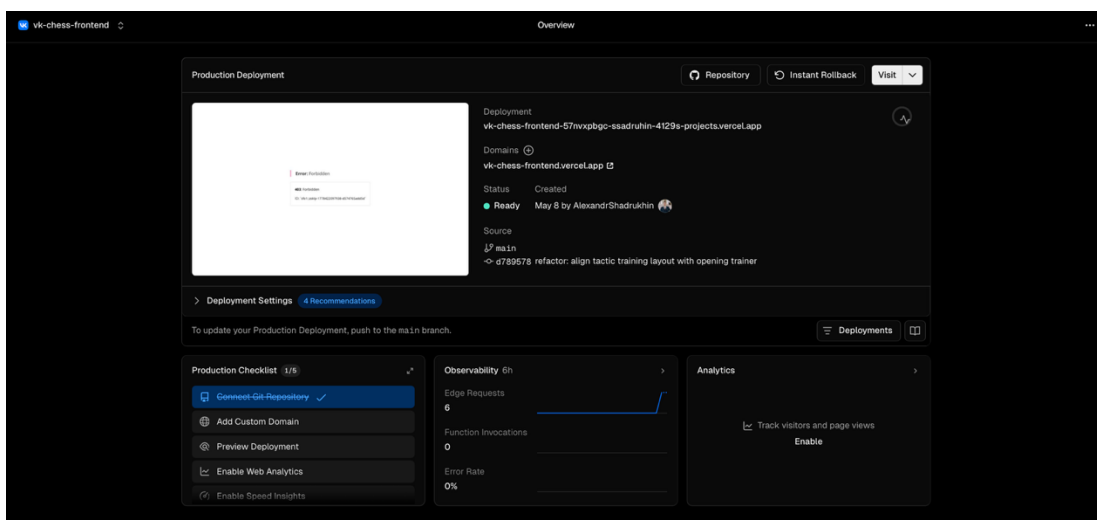


Рисунок 39 – Проверка развертывания frontend-части приложения на платформе Vercel

На рисунке 39 представлена страница развертывания клиентской части приложения. Развертывание выполнено из основной ветки репозитория, статус публикации обозначен как Ready. Это подтверждает, что frontend-часть приложения успешно собрана и доступна для запуска через удаленную платформу.

Серверная часть приложения была развернута на платформе Render [22]. Backend используется для хранения данных пользователя, результатов тестирования, истории прохождения и информации об онлайн-комнатах. На странице Render отображается статус Deployed, что подтверждает успешное развертывание серверной части.

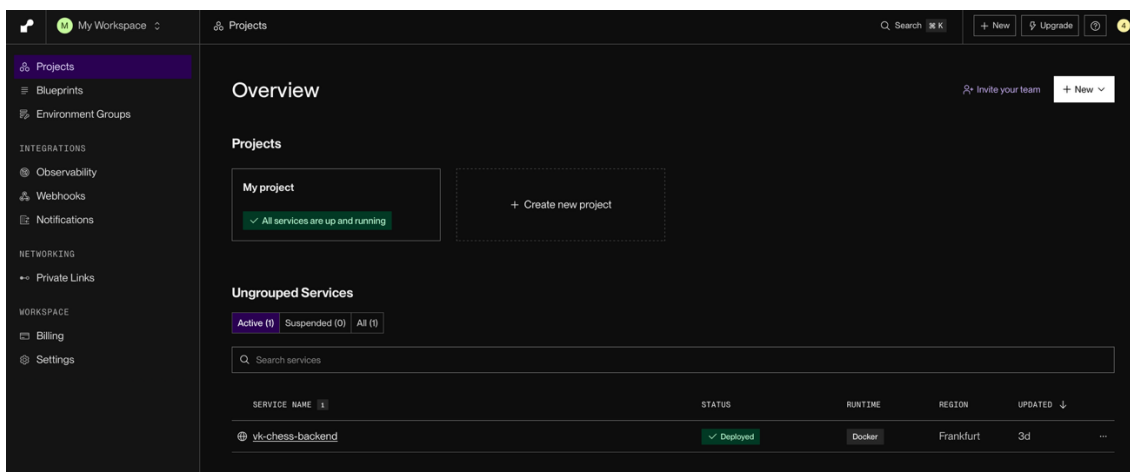


Рисунок 40 – Проверка развертывания backend-части приложения на платформе Render

На рисунке 40 показана панель Render, в которой backend-сервис vk-chess-backend находится в состоянии Deployed. Это подтверждает, что серверная часть приложения успешно развернута и может использоваться клиентской частью для выполнения запросов, связанных с онлайн-игрой, сохранением прогресса и получением результатов тестирования.

Для обобщения результатов функционального тестирования была составлена таблица 2. В таблице приведены основные пользовательские сценарии, которые проверялись в ходе тестирования разработанного веб-приложения. Для каждого сценария указаны ожидаемый результат, фактически полученный результат и итоговый статус проверки.

Такой формат представления результатов позволяет наглядно сопоставить заявленные функциональные возможности приложения с их практической реализацией. В рамках тестирования основное внимание уделялось тем функциям, которые непосредственно связаны с целью выпускной квалификационной работы: обучению шахматным дебютам, прохождению тестов, расчету эффективности обучения, сохранению прогресса пользователя и возможности повторного прохождения сценариев.

Дополнительно были проверены технические аспекты работы приложения: корректность развертывания клиентской части на удаленной платформе, доступность серверной части, а также возможность

взаимодействия frontend- и backend-компонентов при сохранении данных пользователя. Результаты проверки представлены в таблице 2.

Таблица 2 – Результаты функционального тестирования веб-приложения

Проверяемый сценарий	Ожидаемый результат	Фактический результат
Прохождение теста начального уровня	Пользователь отвечает на вопросы, система определяет уровень подготовки	Результат теста сохраняется и отображается в панели прогресса
Прохождение итогового теста	Пользователь выполняет ходы без подсказок, система фиксирует результат	Итоговый результат сохраняется после завершения теста
Расчет эффективности обучения	Система рассчитывает эффективность на основе итогового теста и уровня пользователя	Эффективность обучения отображается в панели прогресса
Сохранение истории прохождения	Система сохраняет начатые и завершенные сценарии	История прохождения отображается в разделе прогресса
Повторное прохождение сценария	Пользователь может продолжить ранее начатый сценарий	Кнопка продолжения доступна в истории прохождения
Развертывание frontend-части	Клиентская часть опубликована на удаленной платформе	Развертывание на Vercel выполнено со статусом Ready
Развертывание backend-части	Серверная часть опубликована на удаленной платформе	Развертывание на Render выполнено со статусом Deployed

По результатам функционального тестирования можно сделать вывод, что основные сценарии работы веб-приложения выполняются корректно. Пользователь может пройти тест начального уровня, выполнить итоговое тестирование, получить рассчитанную оценку эффективности обучения, просмотреть сохраненную историю прохождения и продолжить ранее начатые сценарии. Также была подтверждена работоспособность развернутых frontend- и backend-компонентов приложения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано веб-приложение «ДебютМастер», предназначенное для игры в шахматы и интерактивного обучения шахматным дебютам на платформе VK Games. Разработанное приложение объединяет игровой режим, обучающие сценарии, тестирование уровня подготовки пользователя, итоговую проверку усвоения материала, расчёт эффективности обучения и механизм хранения пользовательского прогресса.

В теоретической части работы был проведён анализ предметной области обучения шахматным дебютам. Были рассмотрены особенности изучения дебютов, существующие подходы к обучению и актуальность использования интерактивных веб-приложений для повышения наглядности и удобства обучения. Также был выполнен обзор существующих аналогов, включая Chess.com, Lichess и шахматные приложения платформы VK Games. По результатам анализа было выявлено, что существующие решения либо ориентированы преимущественно на игровой процесс, либо не предоставляют специализированного интерактивного обучения дебютам с персонализированным отслеживанием прогресса.

В проектной части работы была разработана структура веб-приложения «ДебютМастер». Были построены диаграмма вариантов использования, диаграммы бизнес-процессов, архитектура приложения, структура клиентской и серверной частей, а также ER-диаграмма базы данных. При проектировании была определена клиент-серверная архитектура системы, включающая frontend-часть на React и TypeScript, backend-часть на Java Spring Boot, базу данных PostgreSQL, взаимодействие с платформой VK Games и интеграцию шахматного движка Stockfish.

В практической части работы была реализована клиентская часть приложения с использованием React, TypeScript и VKUI [6-8]. В интерфейсе были реализованы главный экран, игровой раздел, обучающие сценарии, список дебютов, тактические задачи, тест начального уровня, итоговый тест и

панель прогресса пользователя. Для работы с шахматной доской использовались библиотеки `react-chessboard` и `chess.js`, обеспечивающие отображение позиции, обработку ходов и проверку шахматной логики.

Серверная часть приложения была реализована на Java с использованием Spring Boot [14, 15]. Backend-часть обеспечивает создание и управление игровыми комнатами, обработку онлайн-партий, сохранение ходов, хранение прогресса обучающих сценариев, результатов тестирования и эффективности обучения. Для хранения данных использовалась база данных PostgreSQL, а взаимодействие с ней было реализовано через Spring Data JPA. Для онлайн-игры применялось WebSocket-взаимодействие, позволяющее передавать обновления состояния партии между пользователями в режиме реального времени.

В рамках работы был реализован механизм оценки эффективности обучения. Пользователь проходит тест начального уровня, после чего выполняет итоговый тест по изученному дебютному сценарию. На основе результата итогового теста и уровня подготовки пользователя рассчитывается показатель эффективности обучения, который сохраняется в системе и отображается в панели прогресса. Такой подход позволяет учитывать не только абсолютный результат прохождения итогового теста, но и начальный уровень пользователя.

Также была выполнена интеграция шахматного движка Stockfish, позволяющая пользователю играть против компьютерного соперника с различными уровнями сложности. Для онлайн-взаимодействия реализован механизм игровых комнат: пользователь может создать комнату, получить уникальный код и передать его другому игроку для подключения к партии.

Проведённое функциональное тестирование подтвердило корректность работы основных сценариев приложения. Были проверены прохождение теста начального уровня, выполнение итогового теста, расчёт эффективности обучения, сохранение истории прохождения, повторное открытие незавершённых сценариев, а также развертывание клиентской и серверной

частей приложения. Frontend-часть была опубликована на платформе Vercel, а backend-часть — на платформе Render.

Таким образом, цель выпускной квалификационной работы была достигнута: было разработано веб-приложение, направленное на повышение эффективности обучения шахматным дебютам за счёт интерактивных и персонализированных сценариев обучения. Разработанная система позволяет пользователю изучать дебюты пошагово, повторять материал самостоятельно, проходить тестирование, получать оценку эффективности обучения, отслеживать прогресс и применять полученные знания в игровых режимах.

К ограничениям текущей версии приложения можно отнести ограниченный набор дебютных сценариев и тактических задач, а также базовый характер реализованной методики оценки эффективности обучения. В дальнейшем развитие приложения может быть связано с расширением базы дебютов и тренировочных заданий, добавлением более подробной аналитики ошибок пользователя, реализацией системы персонализированных рекомендаций, внедрением рейтингового механизма, улучшением адаптивности интерфейса и проведением апробации методики обучения на группе пользователей.

Исходный код клиентской и серверной частей веб-приложения «ДебютМастер» размещён в открытом репозитории GitHub: <https://github.com/AlexandrShadrukhin/debut-master-vkr>.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Chess.com. Chess Openings and Book Moves [Электронный ресурс]. — URL: <https://www.chess.com/openings> (дата обращения: 15.03.2026).
2. Lichess. Openings [Электронный ресурс]. — URL: <https://lichess.org/opening> (дата обращения: 18.03.2026).
3. VK. Документация для разработчиков VK [Электронный ресурс]. — URL: <https://dev.vk.com/> (дата обращения: 22.03.2026).
4. VK. VK Mini Apps [Электронный ресурс]. — URL: <https://dev.vk.com/mini-apps> (дата обращения: 23.03.2026).
5. VKCOM. VK Bridge: библиотека для интеграции VK Mini Apps с клиентами VK [Электронный ресурс]. — URL: <https://github.com/VKCOM/vk-bridge> (дата обращения: 25.03.2026).
6. VKUI. Документация библиотеки интерфейсных компонентов VKUI [Электронный ресурс]. — URL: <https://vkcom.github.io/VKUI/> (дата обращения: 26.03.2026).
7. Meta. React: библиотека для разработки пользовательских интерфейсов [Электронный ресурс]. — URL: <https://react.dev/> (дата обращения: 30.03.2026).
8. Microsoft. TypeScript Documentation [Электронный ресурс]. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 01.04.2026).
9. Vite. Документация инструмента сборки frontend-приложений [Электронный ресурс]. — URL: <https://vite.dev/guide/> (дата обращения: 03.04.2026).
10. Oakmac. react-chessboard: React-библиотека шахматной доски [Электронный ресурс]. — URL: <https://www.npmjs.com/package/react-chessboard> (дата обращения: 05.04.2026).
11. jhlywa. chess.js: библиотека для реализации шахматной логики [Электронный ресурс]. — URL: <https://github.com/jhlywa/chess.js> (дата обращения: 07.04.2026).

12. Stockfish. Strong open-source chess engine [Электронный ресурс]. — URL: <https://stockfishchess.org/> (дата обращения: 10.04.2026).
13. Stockfish. Stockfish repository [Электронный ресурс]. — URL: <https://github.com/official-stockfish/Stockfish> (дата обращения: 11.04.2026).
14. Oracle. JDK 17 Documentation [Электронный ресурс]. — URL: <https://docs.oracle.com/en/java/javase/17/> (дата обращения: 12.04.2026).
15. Spring. Spring Boot Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/index.html> (дата обращения: 14.04.2026).
16. Spring. Spring Data JPA Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-data/jpa/reference/> (дата обращения: 17.04.2026).
17. Spring. STOMP over WebSocket Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html> (дата обращения: 18.04.2026).
18. PostgreSQL Global Development Group. PostgreSQL Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 20.04.2026).
19. Project Lombok. Lombok features [Электронный ресурс]. — URL: <https://projectlombok.org/features/> (дата обращения: 22.04.2026).
20. Docker. Dockerfile reference [Электронный ресурс]. — URL: <https://docs.docker.com/reference/dockerfile/> (дата обращения: 25.04.2026).
21. Vercel. Vercel Documentation [Электронный ресурс]. — URL: <https://vercel.com/docs> (дата обращения: 05.05.2026).
22. Render. Render Documentation [Электронный ресурс]. — URL: <https://render.com/docs> (дата обращения: 07.05.2026).